

ASTRODYNAMICS

Entraînez votre Agent RL

Introduction à l'Apprentissage par Renforcement

Rafael CEREZO MARTIN

Juin 2026

ASTRODYNAMICS

Brief de Mission
Pilote Automatique pour l'atterrissage d'Eagle-1

1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

Développer un agent RL capable de contrôler le module d'alunissage Eagle-1.
Atterrissage précis, sécurisé et économe en carburant sur zone cible simulée.

PPO · Proximal Policy Optimization | LunarLander-v3 | Stable-Baselines3 | PyTorch

OBJECTIFS TECHNIQUES

1. *Exploration initiale de l'environnement*
Politique aléatoire · inspection obs[8] · baseline −193.6 pts

2. *Choix de l'algorithme et baseline*
DQN discret · PPO général · pipeline fonctionnel

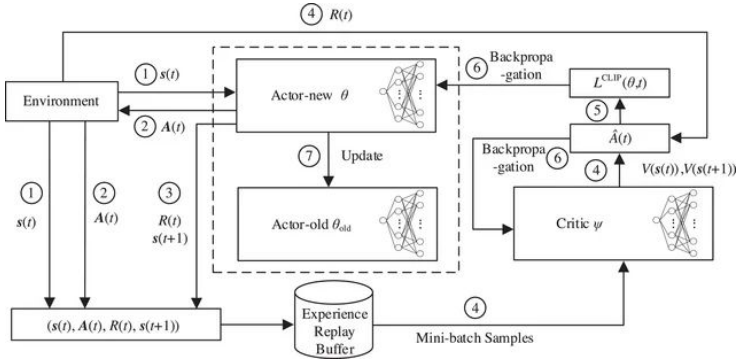
3. *Optimisation des hyperparamètres*
Un paramètre à la fois · TensorBoard · EvalCallback

4. *Évaluation officielle — 100 épisodes*
Politique déterministe · critère ≥ 200 · écart-type faible

5. *Déploiement API + GUI + Dashboard*
FastAPI (8000) · Streamlit (8501) · TensorBoard (6006)

≥ 200 pts · 100 épisodes
Critère officiel

+225.9	61.7	✓ OUI
PPO score moyen	Écart-type	Critère validé



ASTRODYNAMICS

1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

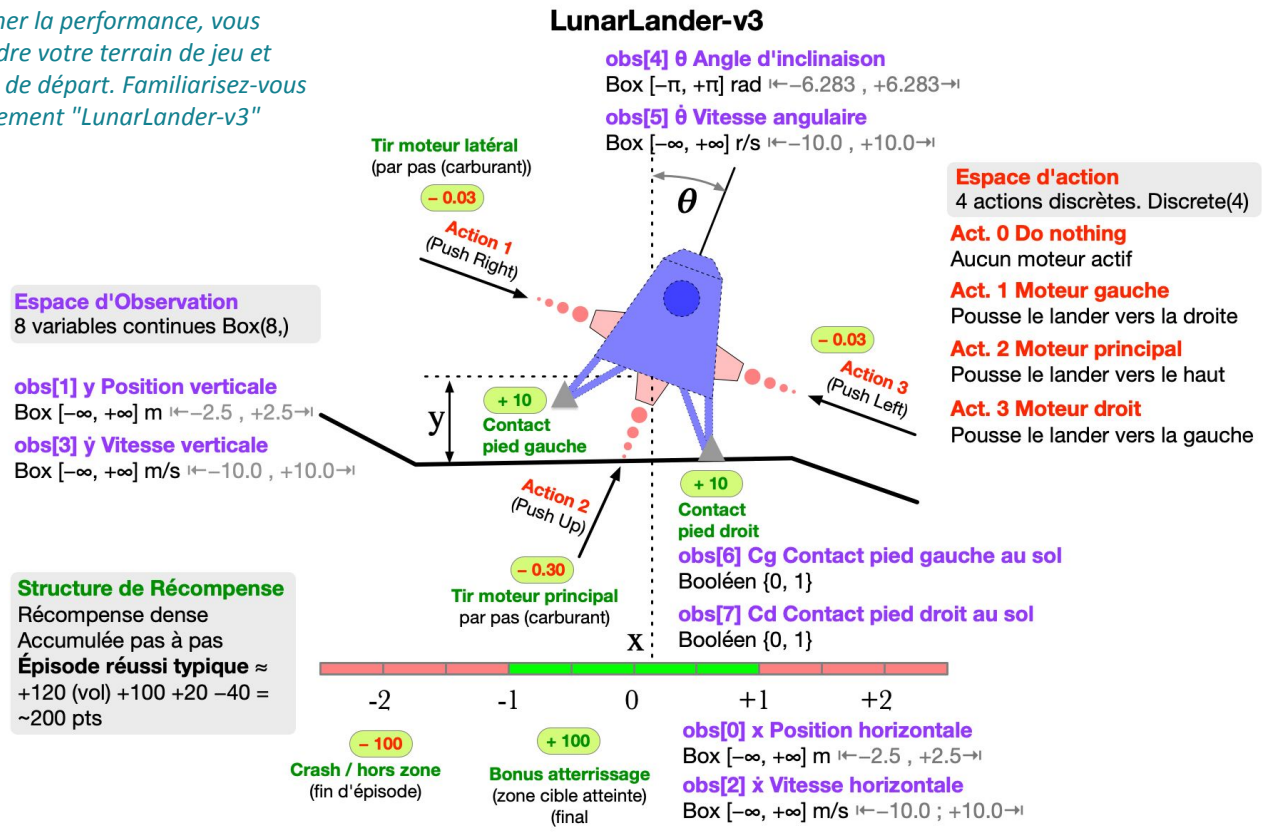
Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

LunarLander-v3

Étape 1 - Explorez l'environnement

Espaces d'observation et d'action (env.observation_space)

Avant de chercher la performance, vous devez comprendre votre terrain de jeu et établir un point de départ. Familiarisez-vous avec l'environnement "LunarLander-v3"



ASTRODYNAMICS

1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

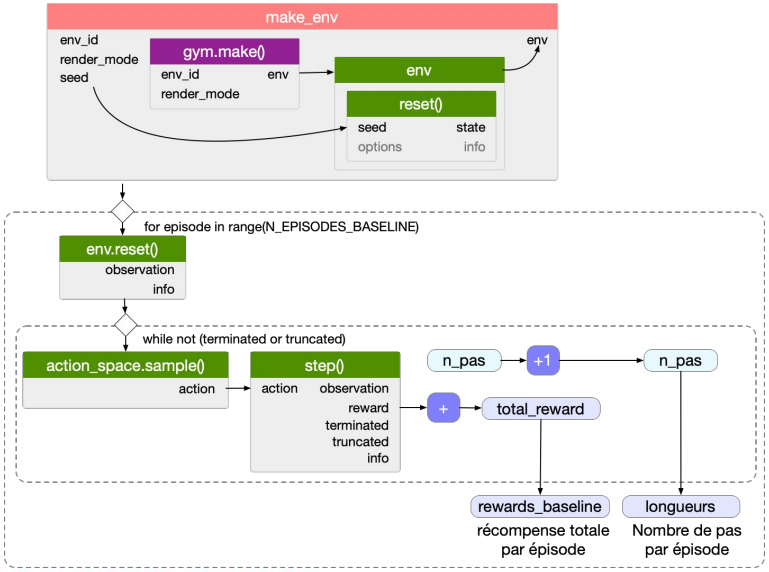
LunarLander-v3

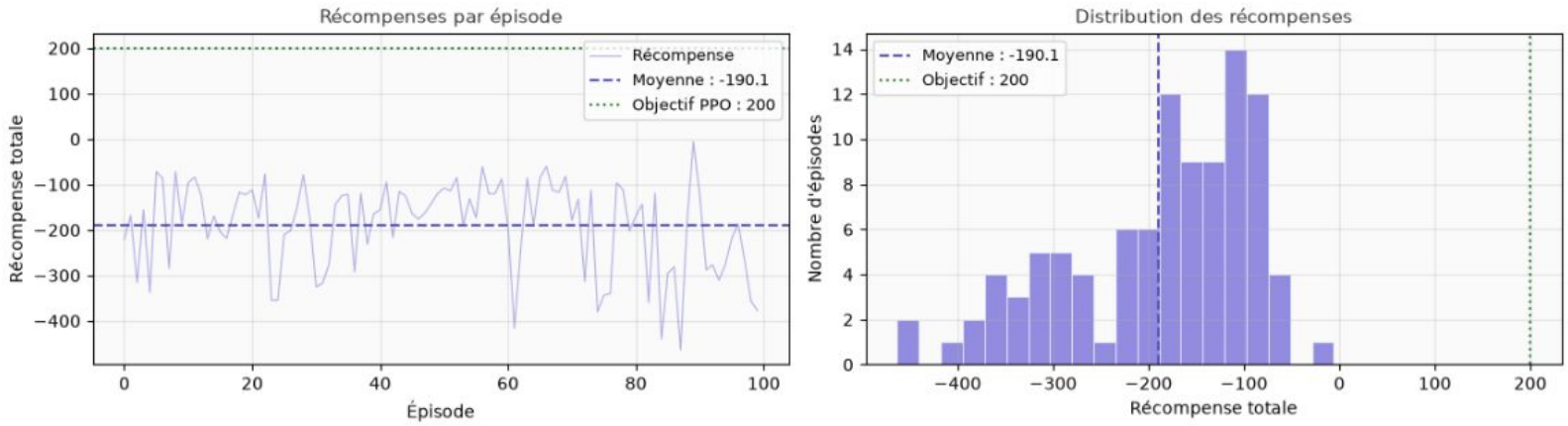
Étape 1 - Etablissez une base de référence

Baseline 100 épisodes

Entraînez un premier agent avec les paramètres par défaut pour mesurer une performance initiale.

```
============================================================
RÉSULTATS BASELINE — POLITIQUE ALÉATOIRE
============================================================
  Récompense moyenne.....: -190.12
  Écart-type.............: 97.74
  Récompense minimale.....: -463.17
  Récompense maximale.....: -5.96
  Durée moyenne (pas).....: 90.4
  Épisodes réussis........: 0 / 100
  Taux de succès..........: 0.0 %
  ──────────────────────────────────────────
  Objectif PPO............: ≥ 200 (écart : +390 pts à combler)
```





ASTRODYNAMICS

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO pour les espaces de contrôle

1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

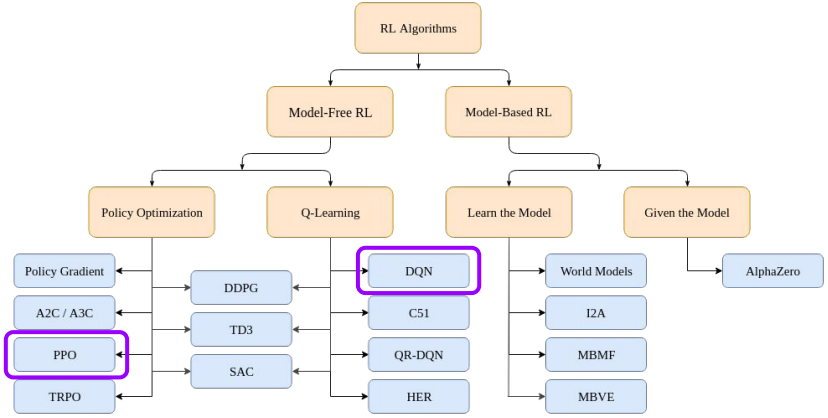
Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

*LunarLander-v3 expose un espace d'action **discret** (`Discrete(4)`).*
DQN et PPO sont tous deux compatibles.
*Nous choisissons **PPO** comme algorithme principal pour deux raisons :*

*1. **Stabilité** : PPO utilise un mécanisme de clipping qui empêche les mises à jour trop agressives de la politique, crucial quand la récompense est dense et variable.*

*2. **Généralité** : PPO fonctionne sur les espaces discrets **et** continus, ce qui le rend réutilisable si l'on passe à la version `continuous=True` de LunarLander*




```
============================================================
CONFIGURATION PPO — EAGLE-1
============================================================
  Environnement...........: LunarLander-v3
  Algorithme..............: PPO (MlpPolicy)
  Total timesteps.........: 500,000

  learning_rate...........: 0.0003
  n_steps.................: 1024
  batch_size..............: 64
  n_epochs................: 10
  gamma...................: 0.999
  gae_lambda..............: 0.98
  clip_range..............: 0.2
  ent_coef................: 0.01
  vf_coef.................: 0.5
  max_grad_norm...........: 0.5

  ── Dérivés ──────────────────────────────
  Mises à jour totales....: 488
  Mini-batchs / update....: 16
  Seuil early stopping....: 250 pts
  Fréquence évaluation....: tous les 10,000 pas

  ── Référence baseline ──────────────────
  Baseline aléatoire......: -193.6 pts (0 % succès)
  Objectif final..........: ≥ +200 pts (100 épisodes)
  Écart à combler.........: +394 pts
```

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO pour les espaces de contrôle

ASTRODYNAMICS

1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

PPO

Architecture acteur-critique

Deux réseaux, π_θ (acteur) choisit l'action · V_φ (critique) estime la valeur

MlpPolicy → acteur + critique partagés

GAE, Generalized Advantage Estimation

Calcul en sens inverse des avantages Â_t = δ_t + γλ·Â_{t+1}

gae_lambda = 0.98

Clipping L^CLIP

Limite les mises à jour · ratio r_t(θ) clampé dans [1−ε, 1+ε]

clip_range = 0.2

Rollout buffer on-policy

Buffer de n_steps transitions collectées avant chaque mise à jour, vidé après

n_steps = 1024

Perte totale composite

L = L^CLIP − c1·L^ENT + c2·L^VF optimisés simultanément

ent_coef=0.01 · vf_coef=0.5

Hyperparamètres génériques

- alpha / learning_rate 3e-4
- gamma 0.999
- epsilon_start *N/A política estocástica*
- epsilon_min / final *N/A política estocástica*
- n_épisodes / timesteps 500 000 steps
- batch_size 64
- seed 42

Hyperparamètres spécifiques

- n_steps 1 024
- n_epochs 10
- gae_lambda 0.98
- clip_range 0.2
- ent_coef 0.01
- vf_coef 0.5
- max_grad_norm 0.5

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO Manuel 1/2 : Étude de l'algorithme PPO. Implémentation manuelle

ASTRODYNAMICS

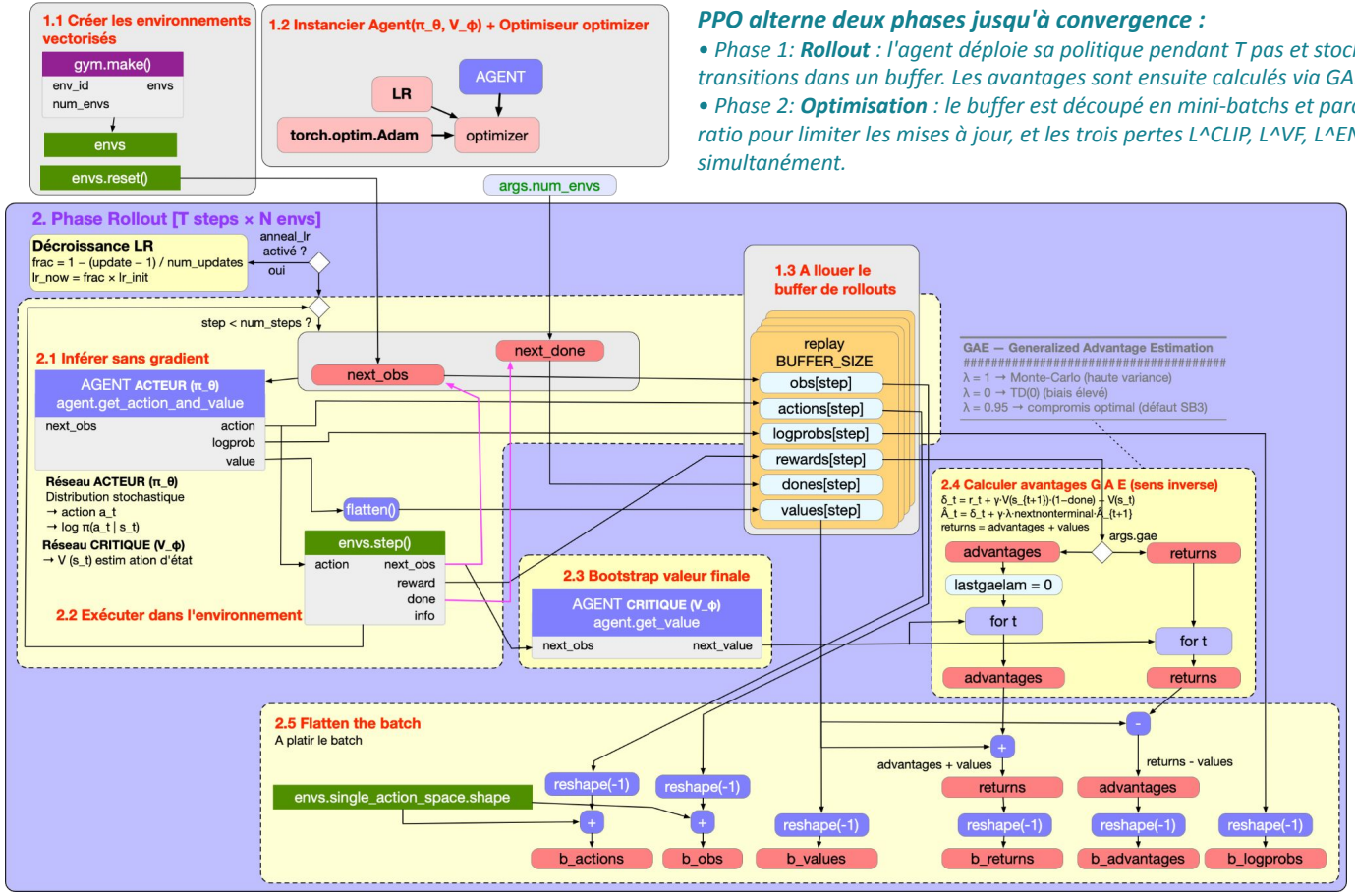
1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)



PPO alterne deux phases jusqu'à convergence :

- *Phase 1: **Rollout** : l'agent déploie sa politique pendant T pas et stocke toutes les transitions dans un buffer. Les avantages sont ensuite calculés via GAE en sens inverse.*
- *Phase 2: **Optimisation** : le buffer est découpé en mini-batchs et parcouru K fois. Le ratio pour limiter les mises à jour, et les trois pertes L^CLIP, L^VF, L^ENT sont optimisées simultanément.*

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO Manuel 2/2 : Entraînement

ASTRODYNAMICS

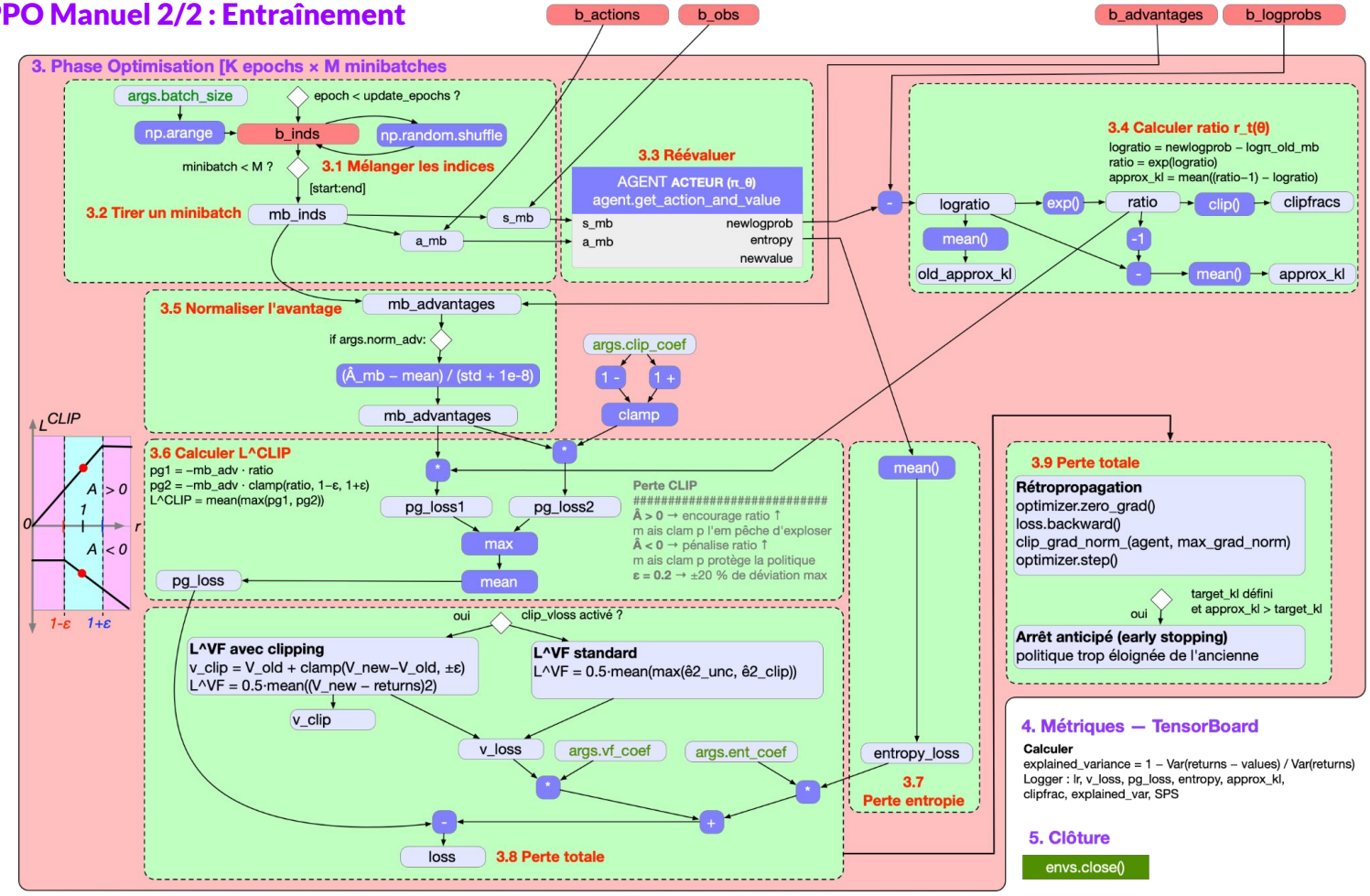
1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)



Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO avec Stable Baselines3 Entraînement

ASTRODYNAMICS

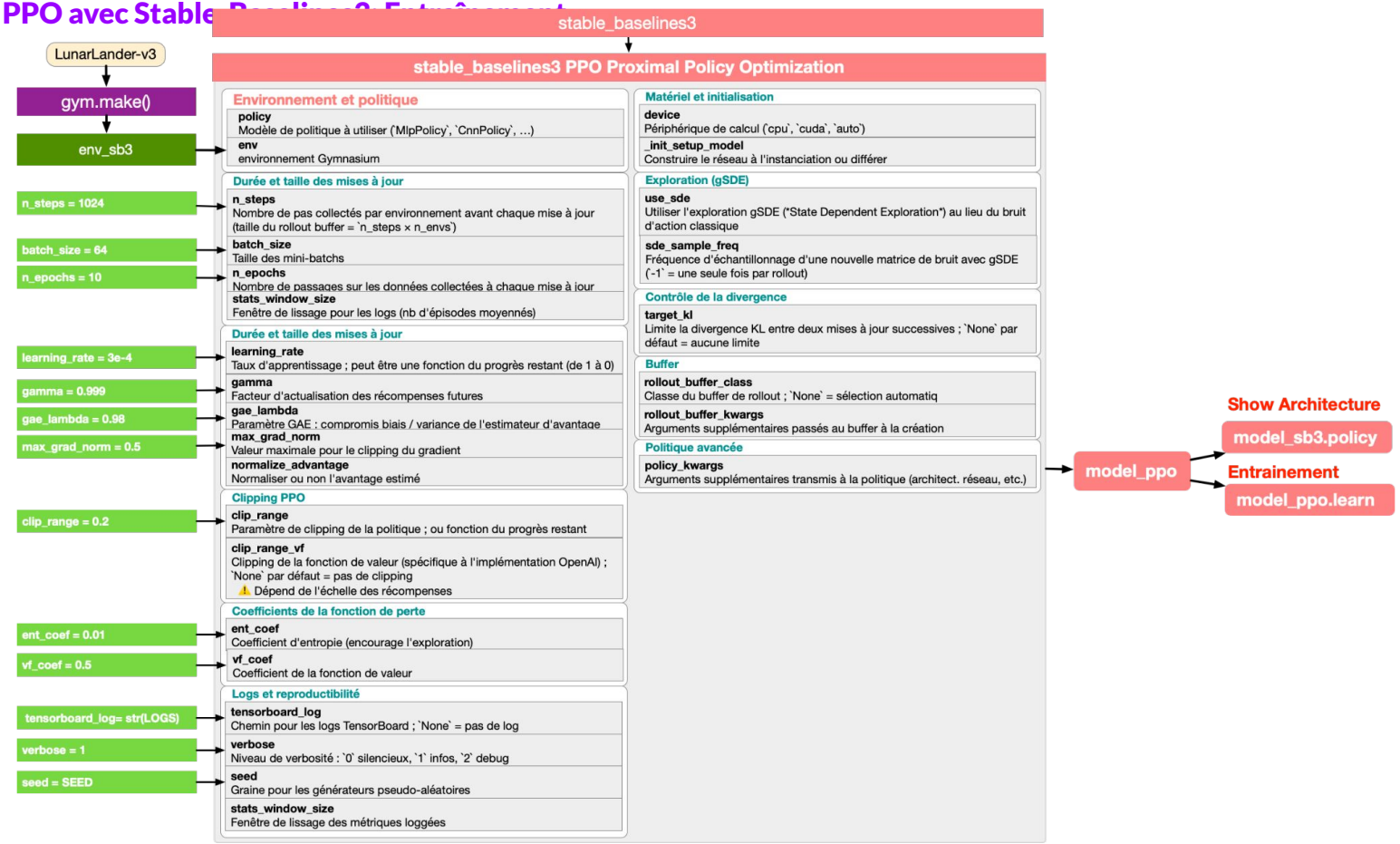
1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)



ASTRODYNAMICS

1

Brief de Mission

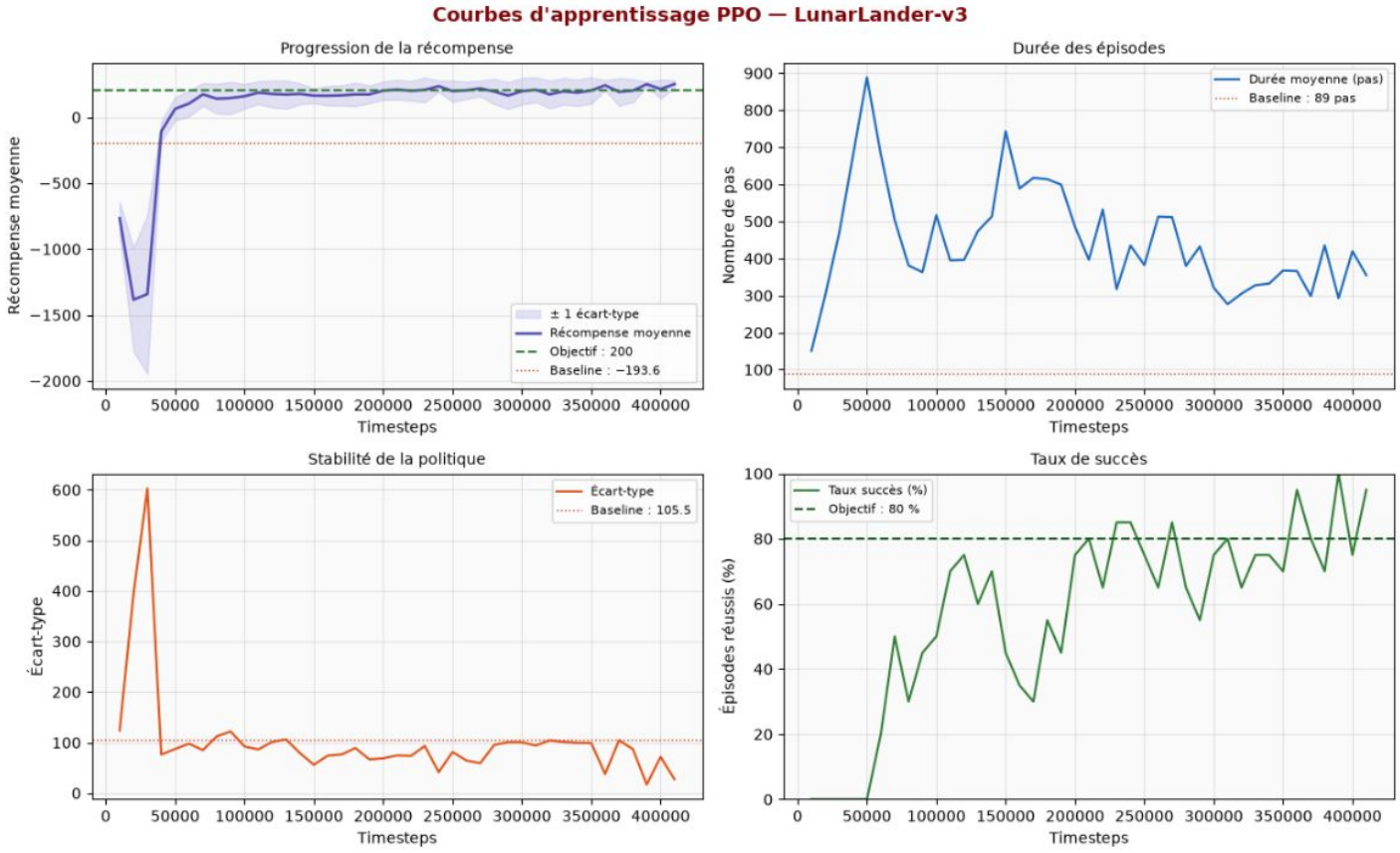
Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO avec Stable-Baselines3: Visualisation



ASTRODYNAMICS

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO avec Stable-Baselines3: Evaluation

1

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

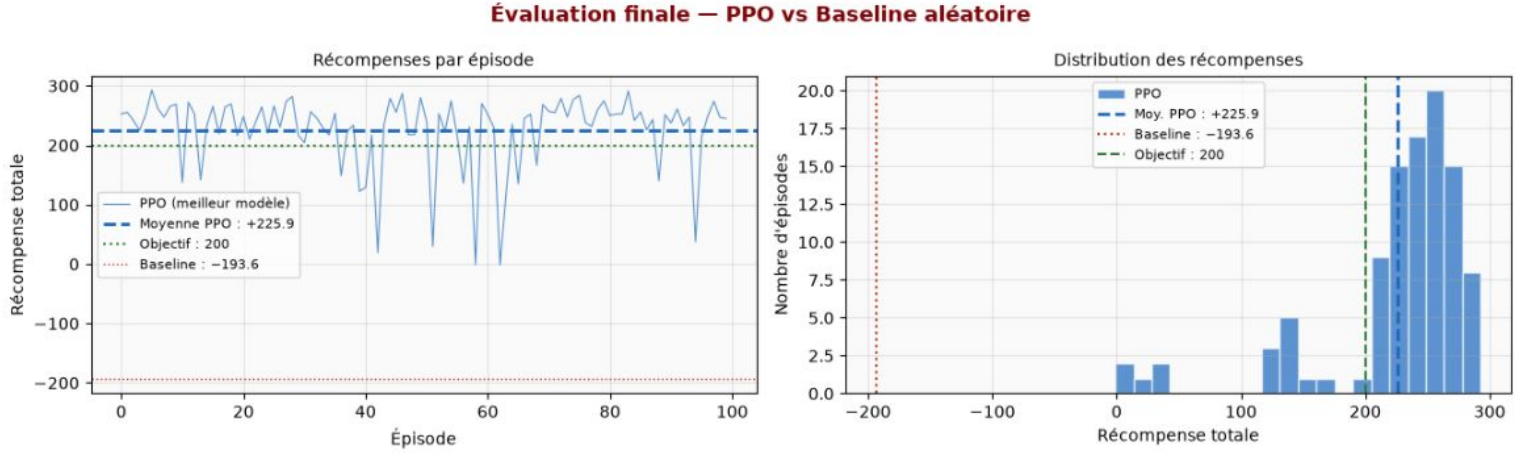
Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

Utiliser la fonction evaluate_policy sur au moins 50 épisodes pour obtenir une mesure de performance de base fiable.

```
============================================================
ÉVALUATION FINALE — MEILLEUR MODÈLE PPO
============================================================
  Modèle chargé...........: m11_ocr\models\best_ppo\best_model.zip
  Récompense moyenne......: +239.62
  Écart-type..............: 54.33
  Critère atteint (≥200)..:✓ OUI
```

```
============================================================
RAPPORT FINAL — 100 ÉPISODES
============================================================
  Récompense moyenne......: +225.88
  Écart-type..............: 61.67
  Récompense minimale.....: -1.22
  Récompense maximale.....: +292.90
  Durée moyenne (pas).....: 387.6
  Épisodes réussis (≥200).: 85 / 100
  Taux de succès..........: 85.0 %
  ──────────────────────────────────────────────────
  Amélioration / baseline.: +419.5 pts
  Critère officiel........:✓ VALIDÉ
  Modèle sauvegardé......:
  m11_ocr\models\mission_260623_151230_ppo_lunarlander_226_best.zip
```



ASTRODYNAMICS

1

Brief de Mission

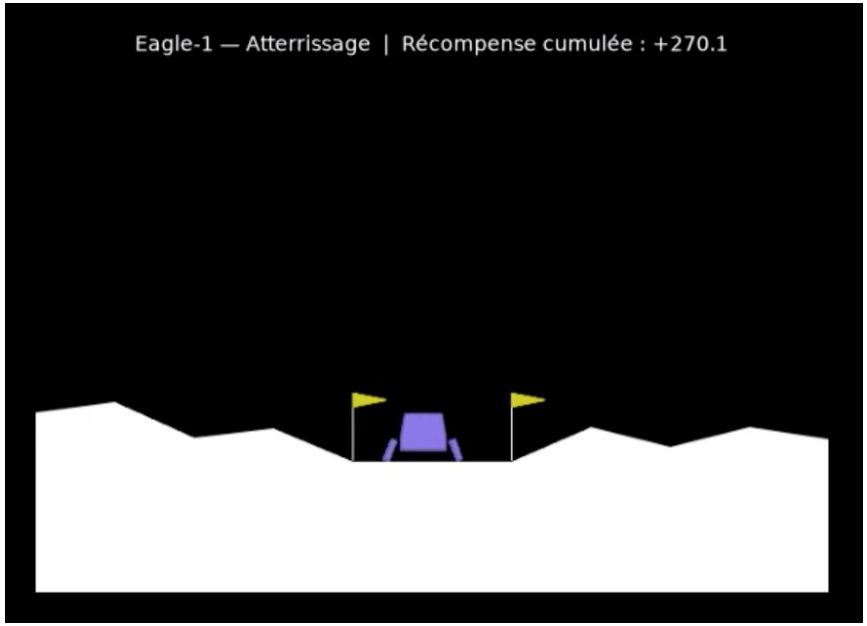
Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

Étape 1 - Adaptez l'algorithme en fonction de l'espace d'action

PPO avec Stable-Baselines3: Animation




```
{
  "projet": "Eagle-1 — AstroDynamics",
  "auteur": "Cerezo_Rafael",
  "date": "260623_005440",
  "fichier_modele":
"mission_260623_005440_ppo_lunarlander_226.zip",
  "environnement": "LunarLander-v3",
  "observation_space": "Box(8,) float32",
  "action_space": "Discrete(4)",
  "algorithme": "PPO",
  "policy": "MlpPolicy",
  "total_timesteps": 500000,
  "hyperparametres": {
    "learning_rate": 0.0003,
    "n_steps": 1024,
    "batch_size": 64,
    "n_epochs": 10,
    "gamma": 0.999,
    "gae_lambda": 0.98,
    "clip_range": 0.2,
    "ent_coef": 0.01,
    "vf_coef": 0.5,
    "max_grad_norm": 0.5
  },
  "resultats": {
    "mean_reward": 225.8817804751369,
    "std_reward": 61.67346832334316,
    "critere_valide": true,
    "n_eval_episodes": 100
  },
  "baseline_aleatoire": {
    "mean_reward": -193.6,
    "std_reward": 105.5,
    "taux_succes": 0.0
  }
}
```

Étape 2 - Optimisez les hyperparamètres de votre agent

ASTRODYNAMICS

2

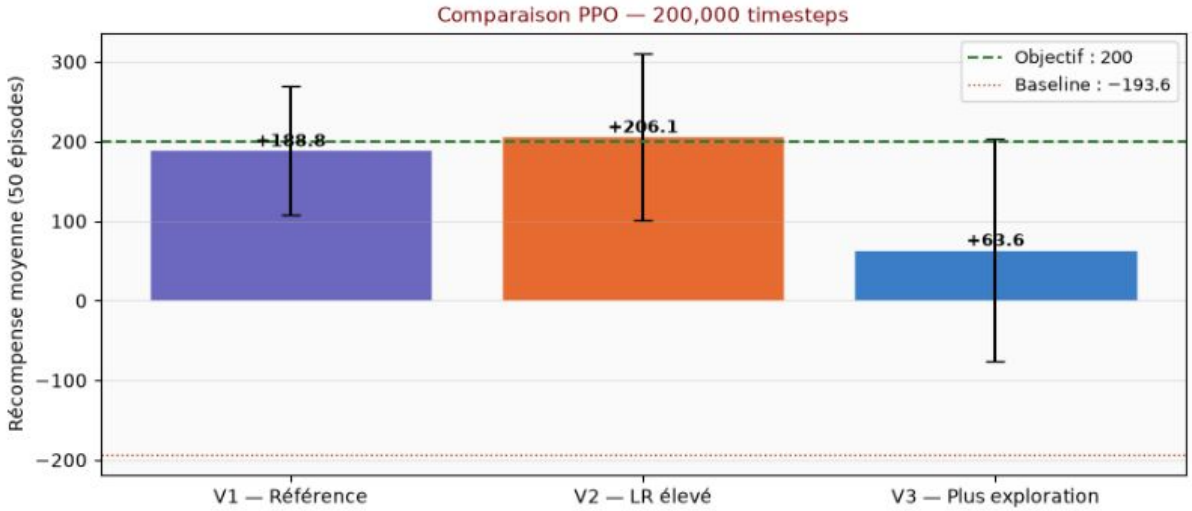
Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

Config	Modification	Hypothèse
V1 (référence)	Hyperparamètres (référence)	Baseline PPO
V2 learning_rate = 1e-3	LR plus élevé	convergence plus rapide ?
V3 ent_coef = 0.05	Plus d'exploration	meilleur score final ?



Étape 2 - Optimisez les hyperparamètres de votre agent

ASTRODYNAMICS

2

Brief de Mission

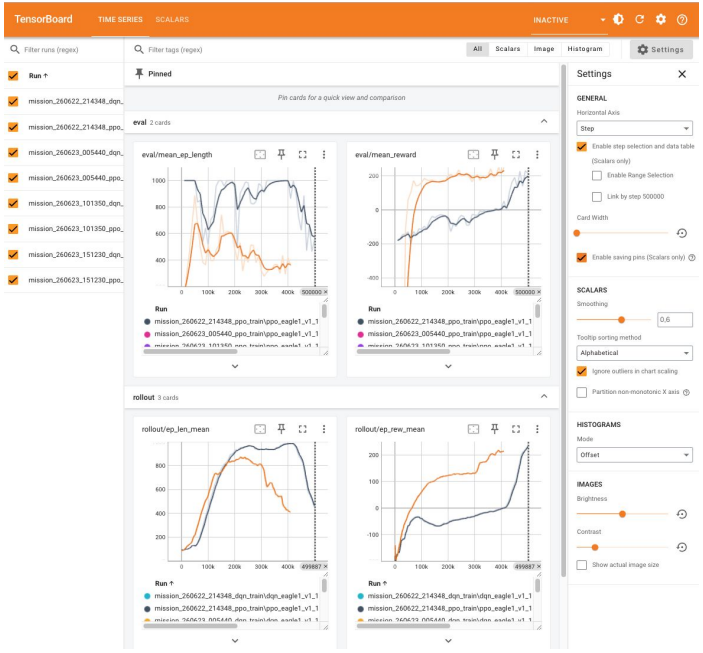
Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

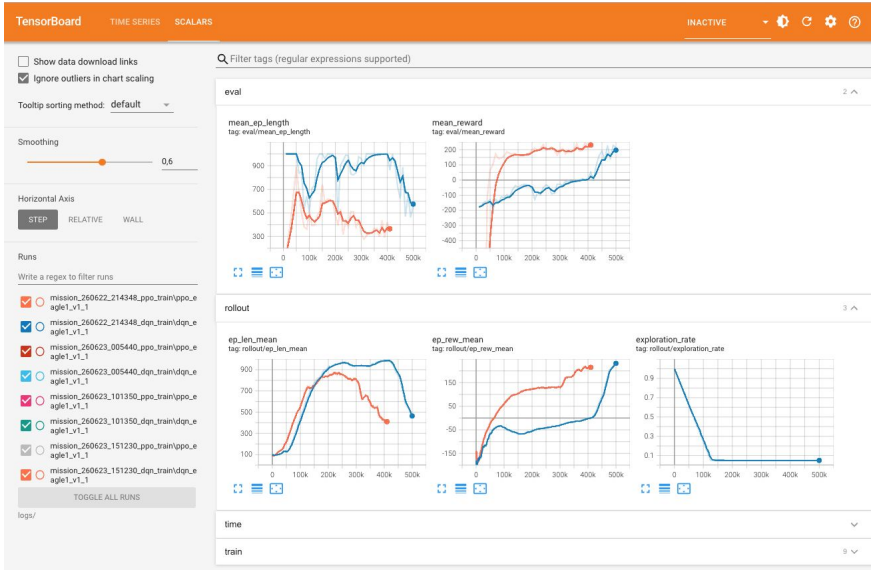
Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

Tensorboard

http://192.168.1.146:6006/#timeseries





ASTRODYNAMICS

2

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

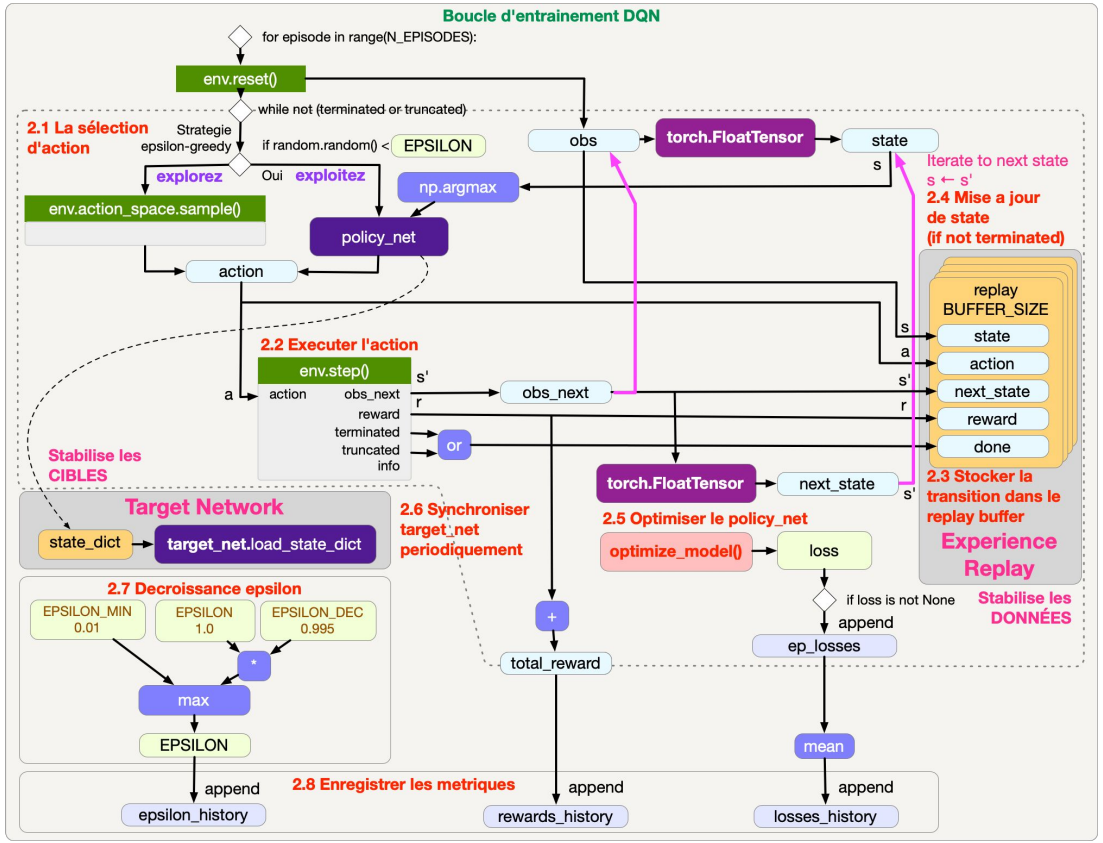
Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

LunarLander-v3

Étape 2 - Optimisez les hyperparamètres de votre agent

Comparaison PPO avec DQN. DQN Manuel



ASTRODYNAMICS

2

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

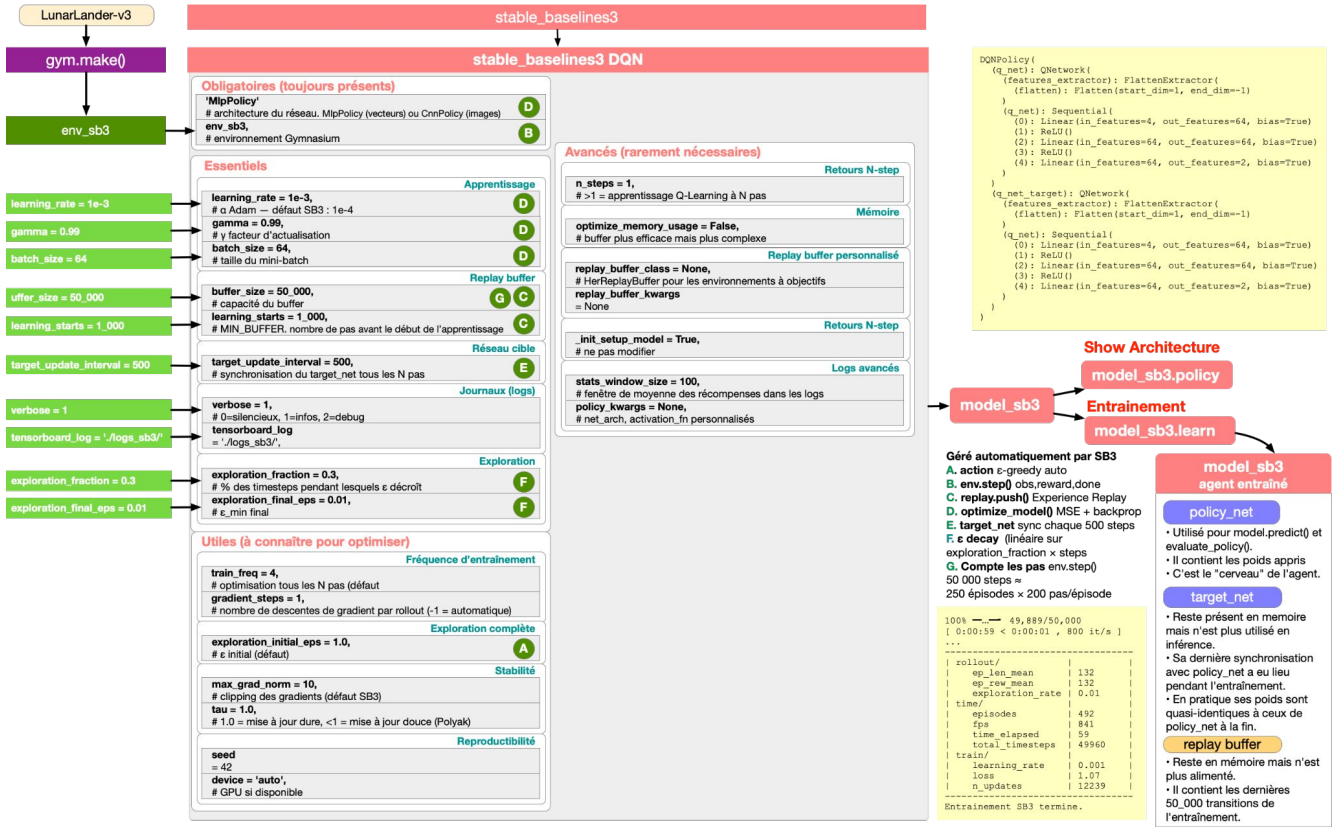
LunarLander-v3

Étape 2 - Optimisez les hyperparamètres de votre agent

Comparaison PPO avec DQN . DQN avec Stable-Baselines3 .

```
============================================================
COMPARAISON PPO vs DQN
============================================================
```

Métrique	PPO	DQN
Récompense moyenne	+225.88	+225.67
Écart-type	61.67	33.38
Critère ≥200 atteint	✓ OUI	✓ OUI
Baseline aléatoire	-193.6	-193.6




```
DQNPolicy(
  (q_net): QNetwork(
    (features_extractor): FlattenExtractor(
      (flatten): Flatten(start_dim=1, end_dim=-1)
    )
    (q_net): Sequential(
      (0): Linear(in_features=4, out_features=64, bias=True)
      (1): ReLU()
      (2): Linear(in_features=64, out_features=64, bias=True)
      (3): ReLU()
      (4): Linear(in_features=64, out_features=2, bias=True)
    )
  )
  (q_net_target): QNetwork(
    (features_extractor): FlattenExtractor(
      (flatten): Flatten(start_dim=1, end_dim=-1)
    )
    (q_net): Sequential(
      (0): Linear(in_features=4, out_features=64, bias=True)
      (1): ReLU()
      (2): Linear(in_features=64, out_features=64, bias=True)
      (3): ReLU()
      (4): Linear(in_features=64, out_features=2, bias=True)
    )
  )
)
```

Show Architecture

model_sb3 → model_sb3.policy / model_sb3.learn (Entrainement)

Géré automatiquement par SB3
- **A. action** ε-greedy auto
- **B. env.step()** obs,reward,done
- **C. replay.push()** Experience Replay
- **D. optimize_model()** MSE + backprop
- **E. target_net** sync chaque 500 steps
- **F. ε decay** (linéaire sur exploration_fraction × steps
- **G. Compte les pas** env.step()

50 000 steps ≈
250 épisodes × 200 pas/épisode

```
100% ━━…━━ 49,889/50,000
[ 0:00:59 < 0:00:01 , 800 it/s ]
...
-------------------------------------
| rollout/                |         |
|    ep_len_mean          | 132     |
|    ep_rew_mean          | 132     |
|    exploration_rate     | 0.01    |
| time/                   |         |
|    episodes             | 492     |
|    fps                  | 841     |
|    time_elapsed         | 59      |
|    total_timesteps      | 49960   |
| train/                  |         |
|    learning_rate        | 0.001   |
|    loss                 | 1.07    |
|    n_updates            | 12239   |
-------------------------------------
Entrainement SB3 termine.
```

model_sb3
agent entraîné

policy_net
- Utilisé pour model.predict() et evaluate_policy().
- Il contient les poids appris
- C'est le "cerveau" de l'agent.

target_net
- Reste présent en memoire mais n'est plus utilisé en inférence.
- Sa dernière synchronisation avec policy_net a eu lieu pendant l'entraînement.
- En pratique ses poids sont quasi-identiques à ceux de policy_net à la fin.

replay buffer
- Reste en mémoire mais n'est plus alimenté.
- Il contient les dernières 50_000 transitions de l'entraînement.

ASTRODYNAMICS

LunarLander-v3

Étape 2 - Optimisez les hyperparamètres de votre agent

Comparaison PPO avec DQN . DQN avec Stable-Baselines3 .

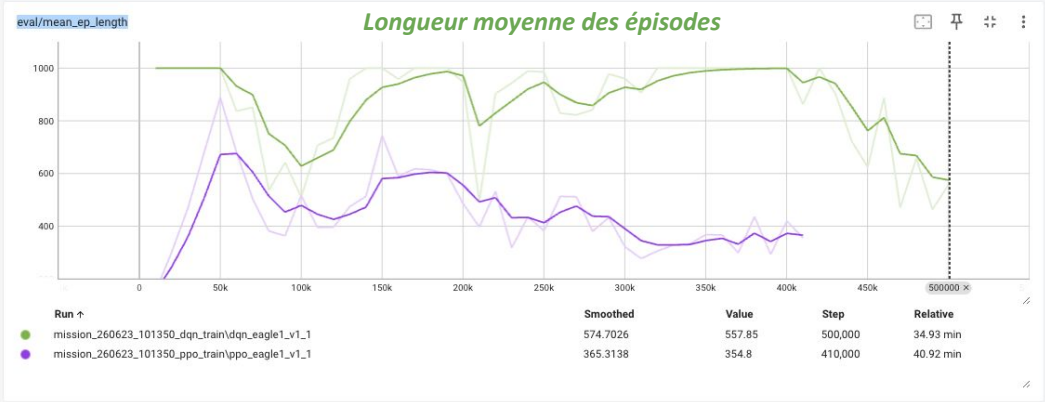
2

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

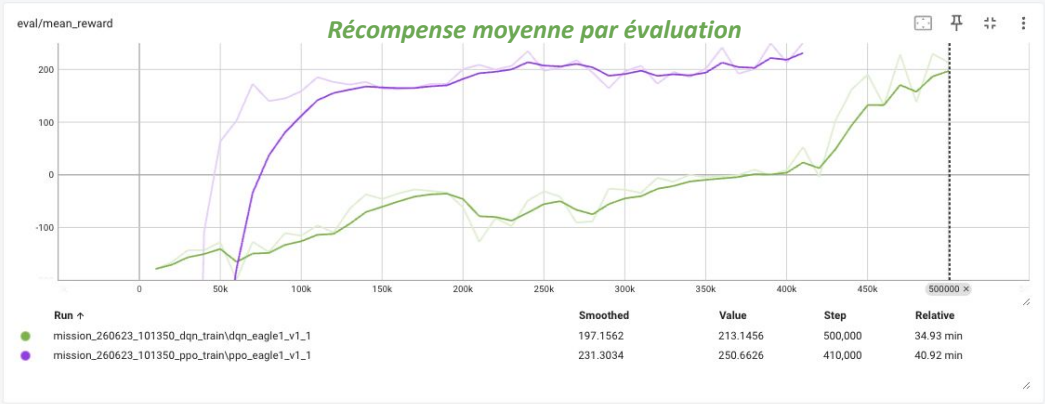
Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)



PPO (violet) descend de 700 vers ~350 pas
L'agent apprend à atterrir de plus en plus vite et directement. Moins de pas = trajectoire plus efficace = moins de carburant consommé = meilleur score.
C'est le signe d'une politique qui converge.

DQN (vert) reste proche de 1000 pas
L'agent ne converge pas vers une décision.
il consomme tout le budget de pas disponible à chaque épisode. Il n'apprend pas à atterrir efficacement, ce qui explique le score moyen plus faible (+196 pts).



PPO (violet) descend de 700 vers ~350 pas
Converge en ~75k steps
Stable après convergence
On-policy → apprend vite
Pas de replay buffer

DQN (vert) reste proche de 1000 pas
Converge en ~450k steps
Continue de progresser
Off-policy → apprend lentement
Replay buffer → retard initial

ASTRODYNAMICS

3

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

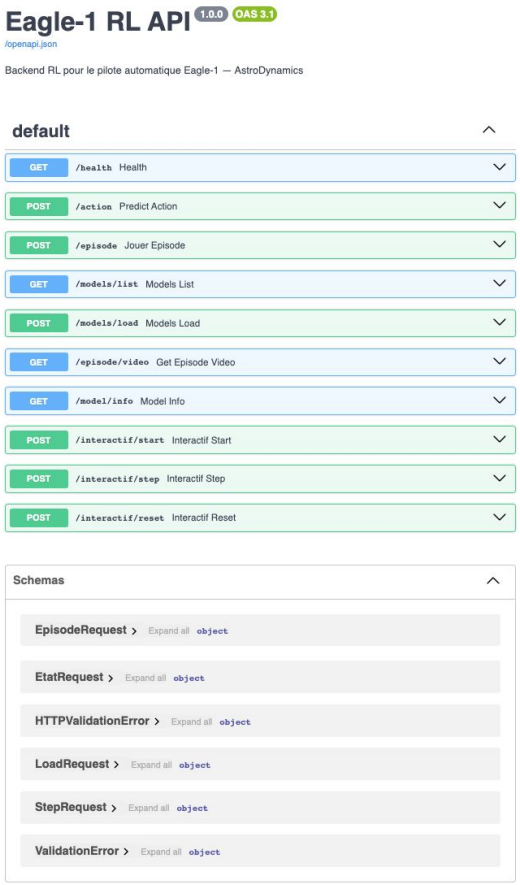
Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

LunarLander-v3

Étape 3 - Déployez une API, une GUI et un tableau de bord

API FastAPI (uvicorn)



L'API FastAPI expose 11 endpoints organisés en 4 groupes :

- ***inférence*** *(/action, /episode),*
- ***gestion des modèles*** *(/models/list, /models/load),*
- ***vidéo*** *(/episode/video) et*
- **mode interactif** (/interactif/start, /step, /reset).

Gestion de modèles

- *Le modèle est chargé une seule fois au démarrage dans un dictionnaire global _etat*
- *Les changements de modèle via* ***/models/load*** *sont instantanés sans redémarrer le serveur.*

Logging

- *Un* ***logger personnalisé*** *RFC 5424 trace chaque endpoint avec horodatage,visible en temps réel dans le terminal uvicorn.*

```
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     192.168.1.108:62648 - "GET /docs HTTP/1.1" 200 OK
INFO:     192.168.1.108:62648 - "GET /openapi.json HTTP/1.1" 200 OK
2026-06-24 15:44:42,625 [api] ##################################
2026-06-24 15:44:42,625 [api]  ┌[API]..............................: [/models/list] ---------------------- [inventaire modeles]
2026-06-24 15:44:42,626 [api]  ┌[API]..............................: [models_list] ---------------------- [scan MODELS/]
2026-06-24 15:44:42,635 [api]          best_ppo.....................: 225.8817804751369
2026-06-24 15:44:42,639 [api]          best_dqn.....................: 225.66695834612577
2026-06-24 15:44:42,645 [api]          ppo mission_260623_183644_ppo_lunarlander_robust: None
2026-06-24 15:44:42,647 [api]          ppo mission_260623_151230_ppo_lunarlander_226_best: None
2026-06-24 15:44:42,649 [api]          ppo mission_260623_151230_ppo_lunarlander_226: 225.8817804751369
2026-06-24 15:44:42,650 [api]          ppo mission_260623_101350_ppo_lunarlander_226_best: None
2026-06-24 15:44:42,652 [api]          ppo mission_260623_101350_ppo_lunarlander_226: 225.8817804751369
2026-06-24 15:44:42,652 [api]          ppo mission_260623_005440_ppo_lunarlander_226_best: None
2026-06-24 15:44:42,655 [api]          ppo mission_260623_005440_ppo_lunarlander_226: 225.8817804751369
2026-06-24 15:44:42,655 [api]          ppo mission_260622_214348_ppo_lunarlander_226_best: None
2026-06-24 15:44:42,657 [api]          ppo mission_260622_214348_ppo_lunarlander_226: 225.8817804751369
2026-06-24 15:44:42,659 [api]          dqn mission_260623_183644_dqn_lunarlander_robust: None
2026-06-24 15:44:42,660 [api]          dqn mission_260623_174837_dqn_lunarlander_robust: None
2026-06-24 15:44:42,660 [api]          dqn mission_260623_151230_dqn_lunarlander_226_best: None
2026-06-24 15:44:42,662 [api]          dqn mission_260623_151230_dqn_lunarlander_226: 225.66695834612577
2026-06-24 15:44:42,663 [api]          dqn mission_260623_151230_dqn_lunarlander_222: None
2026-06-24 15:44:42,664 [api]          dqn mission_260623_101350_dqn_lunarlander_226_best: None
2026-06-24 15:44:42,665 [api]          dqn mission_260623_101350_dqn_lunarlander_226: 225.66695834612577
2026-06-24 15:44:42,666 [api]          dqn mission_260623_101350_dqn_lunarlander_222: None
2026-06-24 15:44:42,666 [api]          dqn mission_260623_005440_dqn_lunarlander_226_best: None
2026-06-24 15:44:42,669 [api]          dqn mission_260623_005440_dqn_lunarlander_226: 225.66695834612577
2026-06-24 15:44:42,669 [api]          dqn mission_260623_005440_dqn_lunarlander_222: None
2026-06-24 15:44:42,670 [api]          dqn mission_260622_214348_dqn_lunarlander_226_best: None
2026-06-24 15:44:42,672 [api]          dqn mission_260622_214348_dqn_lunarlander_226: 225.66695834612577
2026-06-24 15:44:42,672 [api]          dqn mission_260622_214348_dqn_lunarlander_222: None
2026-06-24 15:44:42,672 [api]          total modeles................: 25
2026-06-24 15:44:42,672 [api]  └[API]..............................: [models_list] ---------------------- [25 models]
2026-06-24 15:44:42,672 [api]  └[API]..............................: [/models/list] ---------------------- [25 modeles trouves Exec: 00:00:00.048255]
2026-06-24 15:44:42,672 [api] **********************************
INFO:     192.168.1.108:64938 - "GET /models/list HTTP/1.1" 200 OK
```

ASTRODYNAMICS

3

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

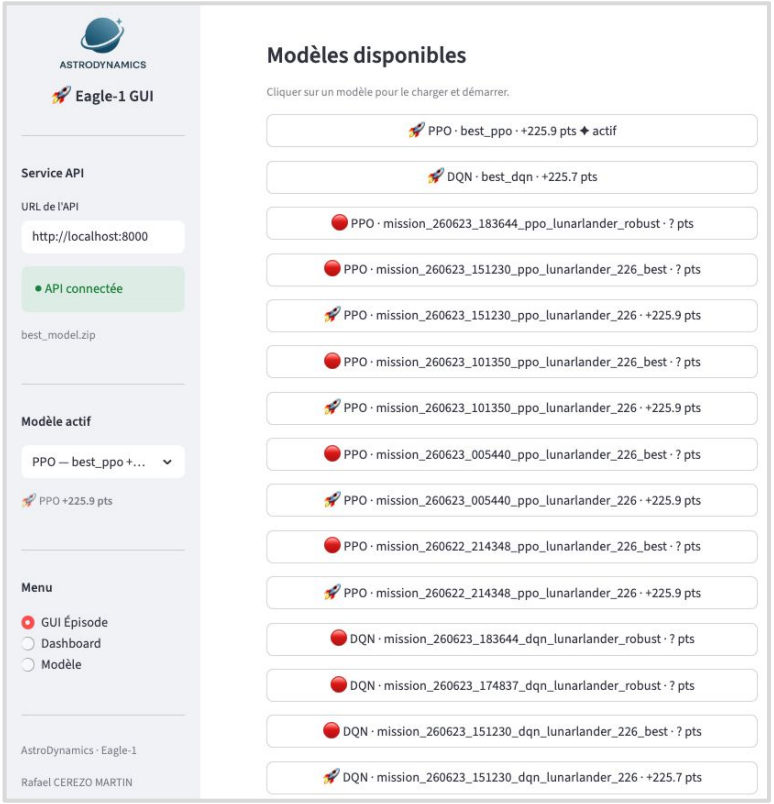
Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

LunarLander-v3

Étape 3 - Déployez une API, une GUI et un tableau de bord

Page GUI Episode (Streamlit)

Seed 44 ✓ Atterrissage réussi — +282.9 pts — 344 pas
Seed 46 ✓ Atterrissage réussi — +261.3 pts — 303 pas
Seed 48 ✓ Atterrissage réussi — +222.4 pts — 300 pas





ASTRODYNAMICS

3

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

LunarLander-v3

Étape 3 - Déployez une API, une GUI et un tableau de bord

Page Dashboard (Streamlit)



Metric cards (4 indicateurs en tête)

*Donnent une lecture instantanée de la **performance globale**:*
épisodes joués, récompense moyenne, taux de succès, meilleur score.

Récompense par épisode (scatter + ligne moyenne)

*Montre la **stabilité de l'agent dans le temps**. Les points verts/rouges distinguent succès et échecs d'un coup d'œil. La ligne pointillée violette matérialise la moyenne.*

Distribution des actions (histogramme filtrable)

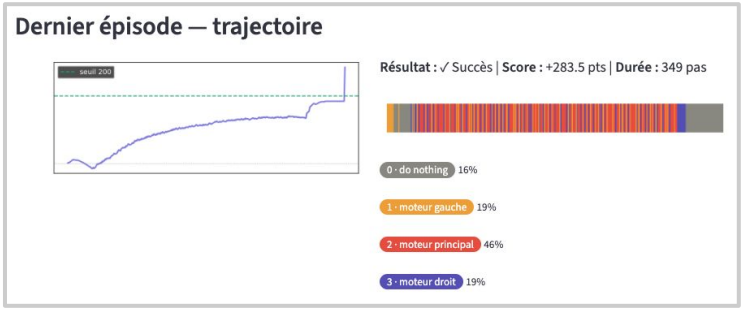
*Révèle la **stratégie apprise par l'agent**: ici 46% moteur principal, ce qui est cohérent avec un atterrissage vertical contrôlé. Le filtre Tous / Succès / Échecs permet de comparer les stratégies gagnantes vs perdantes.*

Décisions selon l'altitude (scatter obs[1] vs action)

*Montre que **l'agent adapte ses décisions à la situation**: moteur principal privilégié à basse altitude, do nothing en phase de descente libre.*
Valide que l'agent a bien appris une politique conditionnelle et non aléatoire.

Dernier épisode: trajectoire + timeline

*Permet **d'analyser un épisode concret** : la courbe de récompense cumulée montre la **progression pas à pas**, la timeline colorée visualise l'enchaînement des actions dans le temps. Utile pour diagnostiquer un crash ou confirmer un atterrissage réussi.*



ASTRODYNAMICS

3

Brief de Mission

Étape 1
- Environnement
- Baseline
- PPO Manuel
- PPO SB3
- Animation

Étape 2
- Hparams. Modification
- Tensorboard
- DQN Manuel
- DQN SB3

Étape 3
- API FastAPI (uvicorn)
- Page Episode (Streamlit)
- Page Dashboard (Streamlit)
- Page Modèle (Streamlit)

LunarLander-v3

Étape 3 - Déployez une API, une GUI et un tableau de bord

Page Modèle (Streamlit)

Informations du modèle

Algorithme	Environnement	Policy
PPO	LunarLander-v3	MlpPolicy

Fichier modèle : `best_model.zip`

Auteur : Cerezo_Rafael | **Date :** 260623_005

Résultats d'évaluation

Récompense moyenne	Écart-type	Critère ≥ 200
+225.9	61.7	✓ Validé

Évaluation sur **100 épisodes**.

Hyperparamètres PPO

Paramètre	Valeur
learning_rate	0.0003
n_steps	1024
batch_size	64
n_epochs	10
gamma	0.999
gae_lambda	0.98
clip_range	0.2
ent_coef	0.01
vf_coef	0.5
max_grad_norm	0.5

Identification du modèle (algorithme, environnement, policy)

*Permet de vérifier en un coup d'œil quel **modèle est actif:** PPO / DQN, sur quel **environnement**, avec quelle **architecture réseau**. Utile quand plusieurs modèles sont disponibles et que l'on change de modèle via le sélecteur.*

Résultats d'évaluation (3 métriques)

*Trois indicateurs: **récompense moyenne**, **écart-type (stabilité)**, et **validation du seuil ≥ 200** sur 100 épisodes. L'écart-type est aussi important que la moyenne: un agent qui atteint 225 pts avec σ=61 est plus fiable qu'un agent à 230 pts avec σ=120.*

Table des hyperparamètres PPO

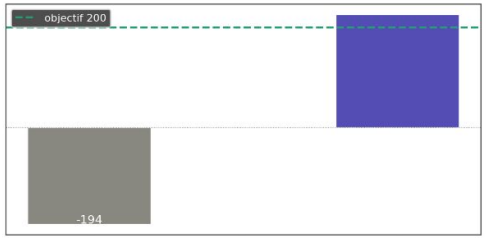
Documente les choix techniques ayant mené au résultat:
- *`**gamma**=0.999` favorise la planification à long terme,*
- *`**gae_lambda**=0.98` réduit le biais des estimations d'avantage,*
- *`**clip_range**=0.2` est la valeur standard PPO.*

Comparaison des agents (table + graphique)

La progression de −193.6 (baseline aléatoire) à +225.9 (PPO) quantifie l'apport du RL. Le graphique en barres rend la comparaison immédiatement lisible.

Comparaison des agents

Agent	Récompense moyenne	Écart-type	Critère ≥ 200
Baseline aléatoire	-193.6	105.5	✗
DQN	—	—	✗
PPO	225.8817804751369	61.67346832334316	✓



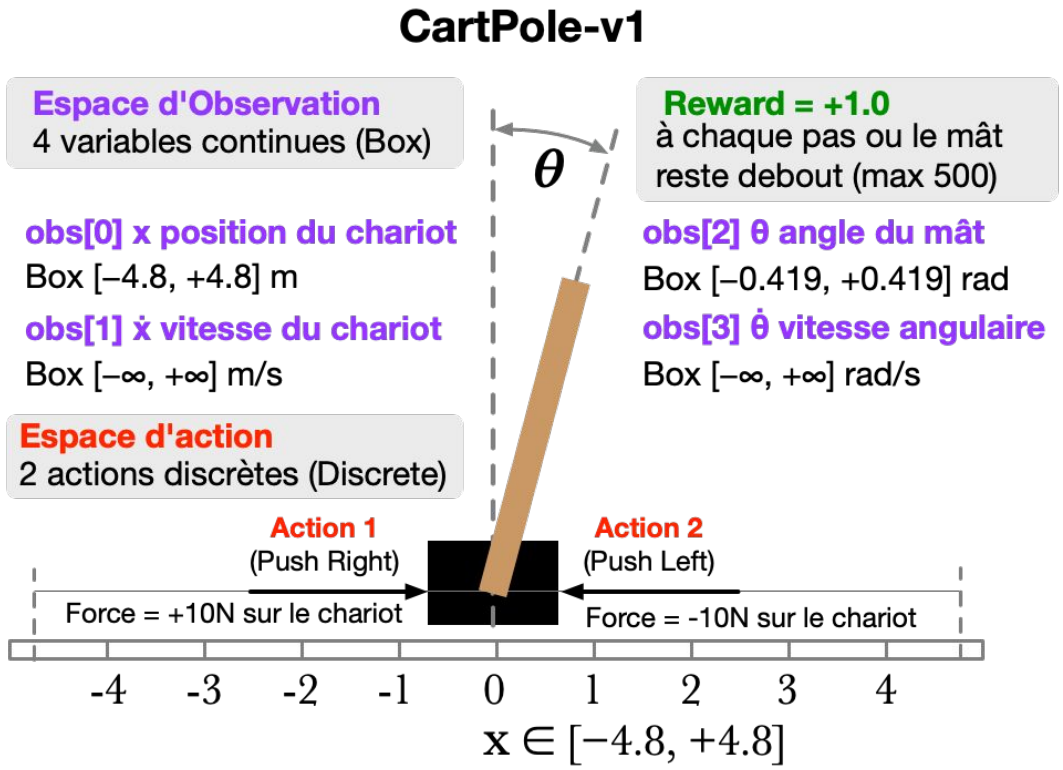


Q et R

Rafael CEREZO MARTIN

Juin 2026

Découvrez les blocs de construction de l'Apprentissage par Renforcement



Découvrez les blocs de construction de l'Apprentissage par Renforcement

ASTRODYNAMICS

1

Introduction

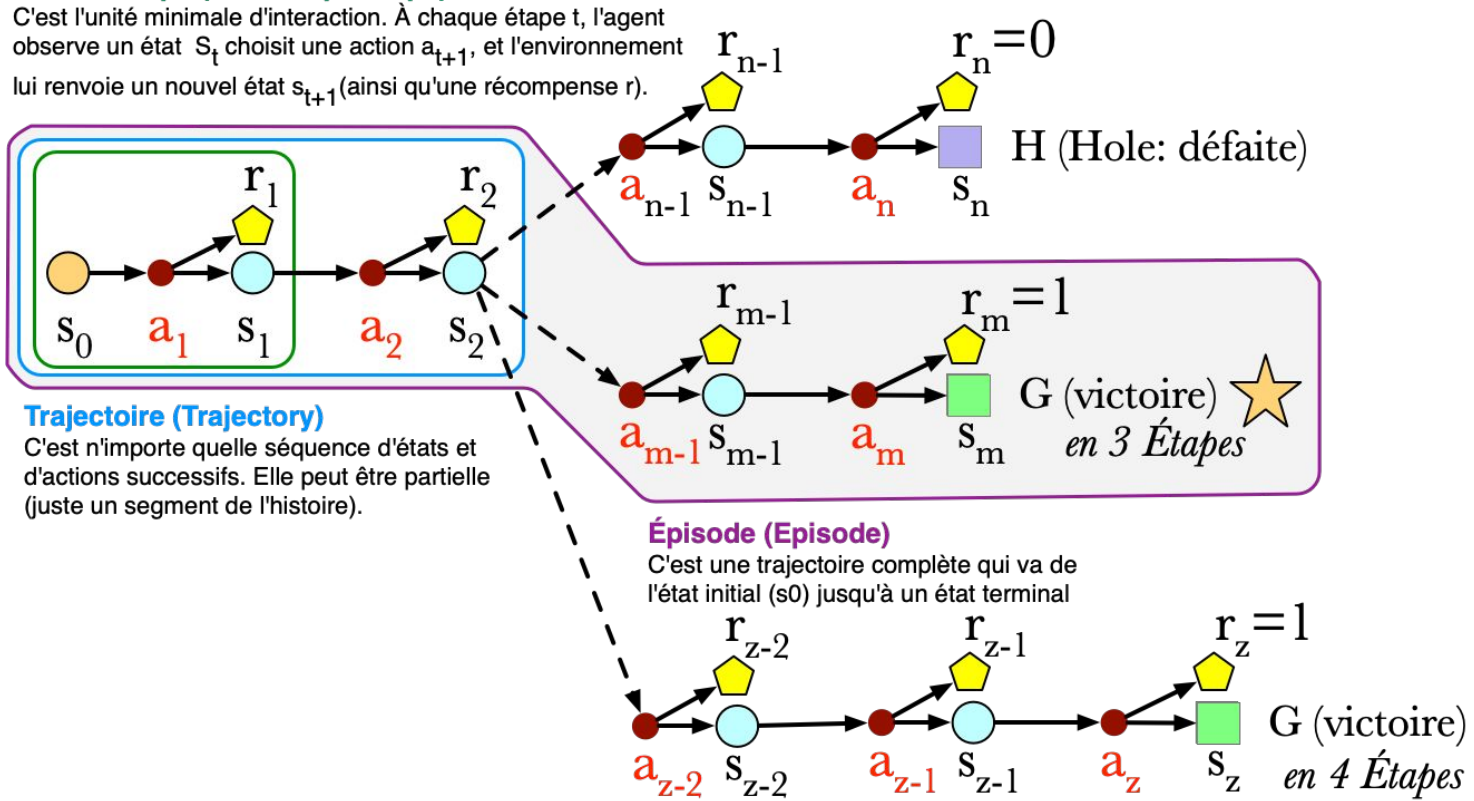
Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Pas de temps (Time-step / Étape)

C'est l'unité minimale d'interaction. À chaque étape t, l'agent observe un état S_t choisit une action a_{t+1}, et l'environnement lui renvoie un nouvel état s_{t+1}(ainsi qu'une récompense r).



Trajectoire (Trajectory)

C'est n'importe quelle séquence d'états et d'actions successifs. Elle peut être partielle (juste un segment de l'histoire).

Épisode (Episode)

C'est une trajectoire complète qui va de l'état initial (s0) jusqu'à un état terminal

Découvrez les blocs de construction de l'Apprentissage par Renforcement

ASTRODYNAMICS

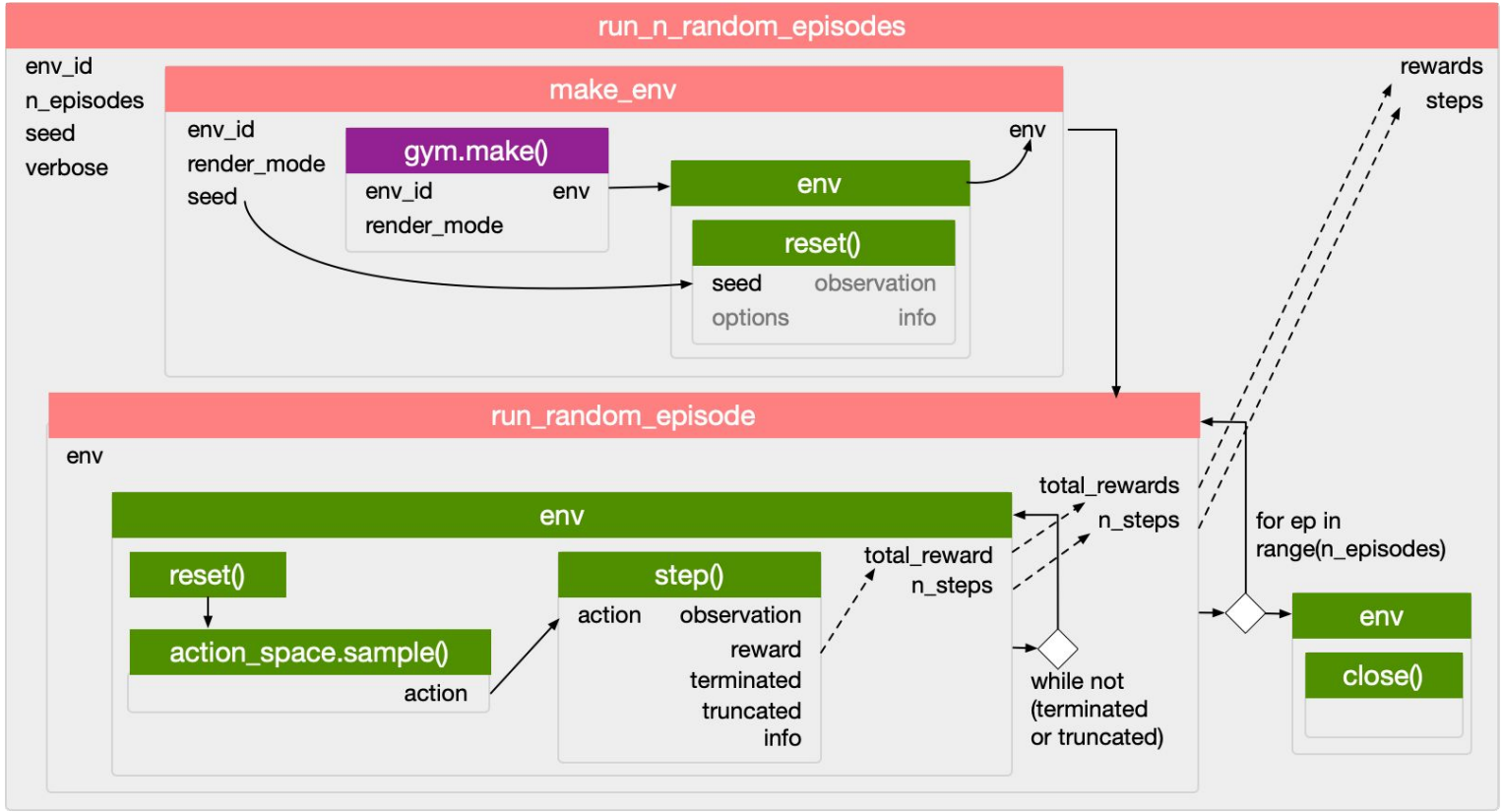
1

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3



Découvrez les blocs de construction de l'Apprentissage par Renforcement

1

Introduction

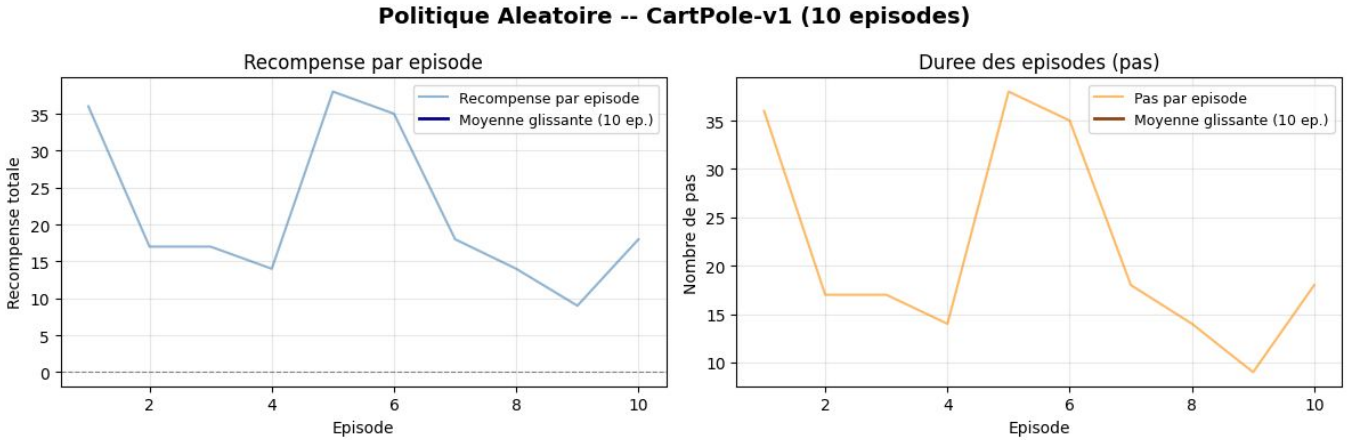
Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Ces 10 épisodes constituent le baseline

La référence minimale que tout agent entraîné doit surmonter.

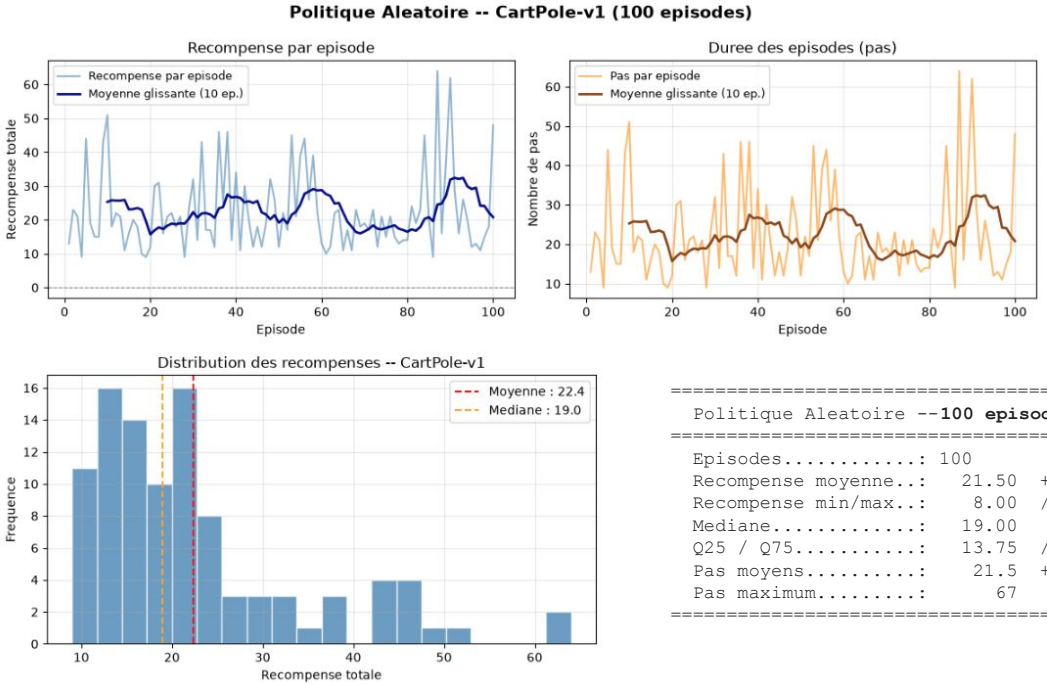



```
================================================
  Politique Aleatoire --10 episodes -- CartPole-v1
================================================
  Episodes.............: 10
  Recompense moyenne..:    21.60  +-   9.99
  Recompense min/max..:     9.00  /    38.00
  Mediane.............:    17.50
  Q25 / Q75...........:    14.75  /    30.75
  Pas moyens..........:     21.6  +-   10.0
  Pas maximum.........:       38
================================================
```

Interprétation (10 episodes)

- *Variance tres elevee : chaque épisode dure entre 10 et 150 pas selon la chance*
- *Récompense = nombre de pas dans CartPole (+1 par pas ou le mat reste debout)*
- *10 épisodes insuffisants pour une estimation fiable — passage a 100*

Découvrez les blocs de construction de l'Apprentissage par Renforcement




```
================================================
  Politique Aleatoire --100 episodes -- CartPole-v1
================================================
  Episodes.............: 100
  Recompense moyenne..:   21.50  +-  11.06
  Recompense min/max..:    8.00  /   67.00
  Mediane.............:   19.00
  Q25 / Q75...........:   13.75  /   25.00
  Pas moyens..........:    21.5  +-  11.1
  Pas maximum.........:      67
================================================
```

Interprétation (10 episodes)

- *Récompense moyenne 22.4 ± 11.8 — aucune stratégie, résultat purement aléatoire*
- *Distribution concentrée entre 10 et 25 pas — forte variabilité, aucune stabilité*
- *Médiane (19) < moyenne (22.4) — quelques épisodes chanceux tirent la moyenne vers le haut*
- *Courbe sans tendance — aucun apprentissage, chaque épisode est indépendant*
- *Établit la ligne de base : tout agent entraîné doit dépasser ces 22 pts*

Découvrez les blocs de construction de l'Apprentissage par Renforcement

ASTRODYNAMICS

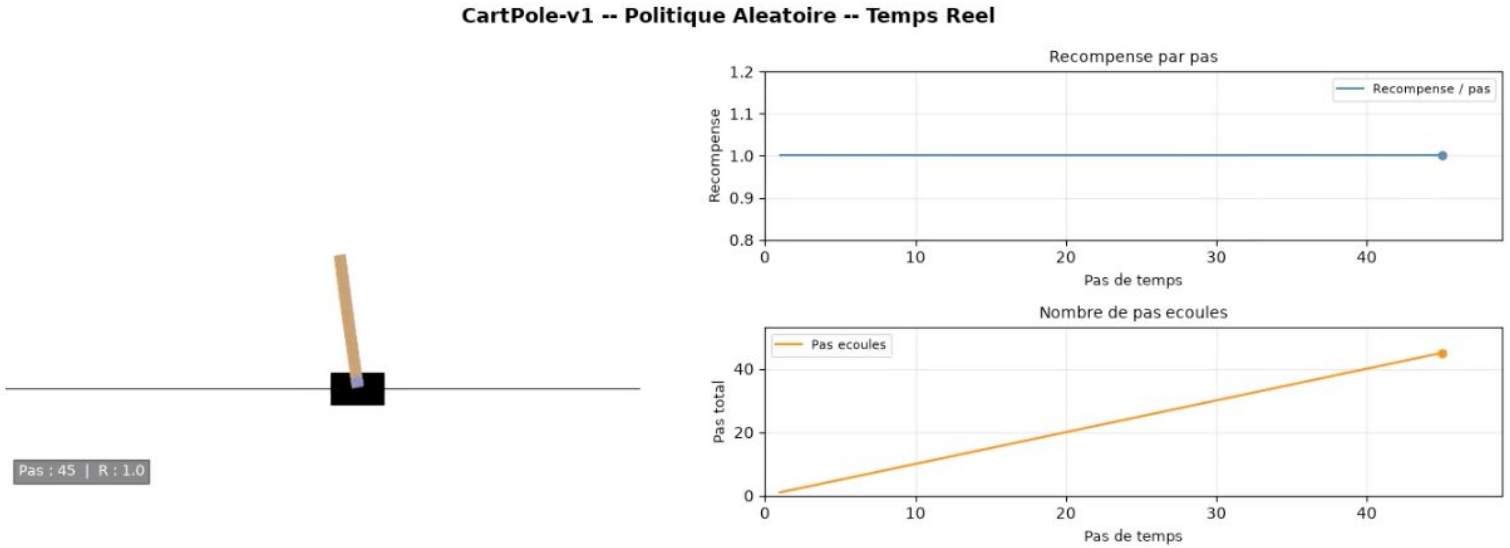
1

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3



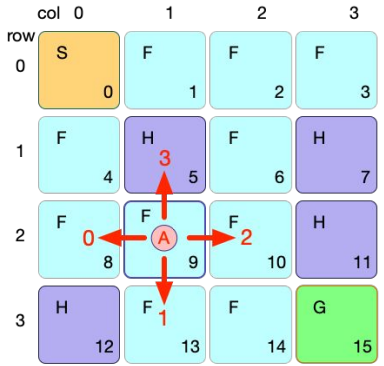
Entraînez un agent avec une table de décision (Q-table)

FrozenLake-v1

Espace d'Observation
Discrete(16) états
Type : Discrete (entier 0..15)
Codage : état = row × 4 + col
reset() : obs = 0 (toujours S)
step() : obs ∈ {0..15}
sample() : entier aléatoire ∈ {0..15}

Espace d'action
Discrete(4) actions
0 ← LEFT
1 ↓ DOWN
2 → RIGHT
3 ↑ UP

is_slippery=True (défaut)



Grille 4×4 (défaut)
With Parameter:
map_name='8x8' → 64 états

Reward ∈ (0,1)
Récompense
+1.0 arriver sur G (état 15)
0.0 tous les autres pas
0.0 tomber dans H

Terminaison
True → G ou H
False → autres pas

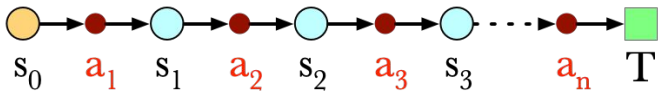
gym.make('FrozenLake-v1', is_slippery= **?**)
is_slippery=True : probabilité stochastique / probabiliste
→1/3 direction voulue,
→2/3 perpendiculaires
action réelle ≠ action choisie
is_slippery= False
déterministe (sans hasard)

Début — état 0
reset() → ici toujours

Gelé — sol sûr
récompense = 0, continue

Hole — trou (défaite)
reward = 0, terminated

Goal — but (victoire)
reward = +1, terminated



Entraînez un agent avec une table de décision (Q-table)

Q-Learning

Q-table

Matrice NumPy (n_états × n_actions), représentation tabulaire de Q(s,a)

np.zeros((16, 4))

Mise à jour Bellman directe

Pas de réseau, la Q-table est mise à jour à chaque pas en place

q[s,a] += α·(r + γ·max(q[s']) − q[s,a])

Décroissance ε multiplicative

ε réduit à chaque épisode par un facteur fixe

ε = max(ε·decay, ε_min)

Espace d'états discret requis

Ne fonctionne que si les états peuvent indexer la table, FrozenLake (16 états)

Hyperparamètres génériques

- alpha learning_rate 0.1
- gamma 0.99
- epsilon_start 1.0
- epsilon_min / final 0.01
- n_épisodes / timesteps 10 000 ep
- batch_size N/A
- seed 42

Hyperparamètres spécifiques

- epsilon_decay 0.995
- n_episodes 10 000
- n_eval_episodes 100

Entraînez un agent avec une table de décision (Q-table)

ASTRODYNAMICS

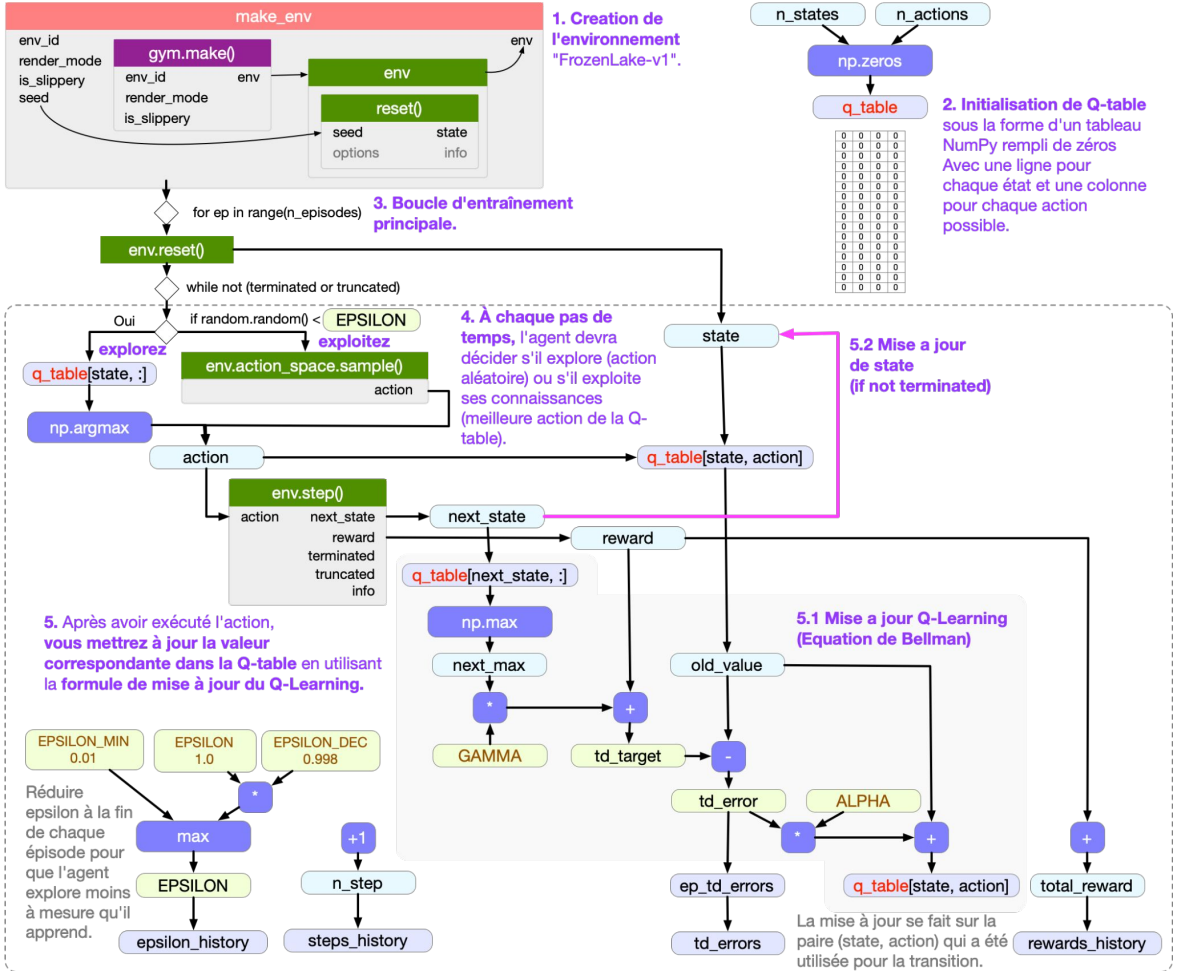
2

Introduction

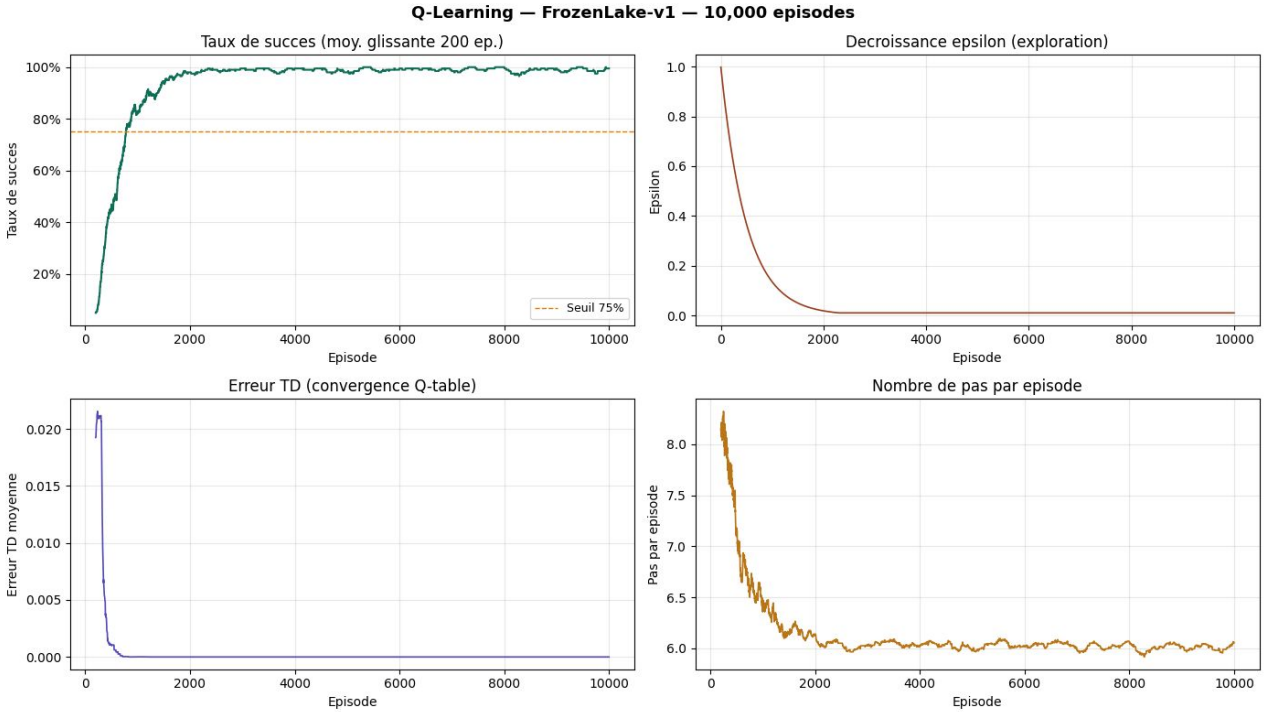
Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3



Entraînez un agent avec une table de décision (Q-table)



Interprétation (10 episodes)

- *Taux de succes : doit monter de ~0% vers 70-80% apres convergence*
- *Epsilon : decroit lineairement de 1.0 vers 0.01 — l'agent explore de moins en moins*
- *Erreur TD : doit diminuer — la Q-table converge vers les vraies valeurs*
- *Pas par episode : peut augmenter puis stabiliser — l'agent trouve des chemins plus efficaces*

Note : is_slippery=True (defaut) rend la convergence difficile. Taux de succes de 70-75% est excellent dans ce cas

ASTRODYNAMICS

2

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

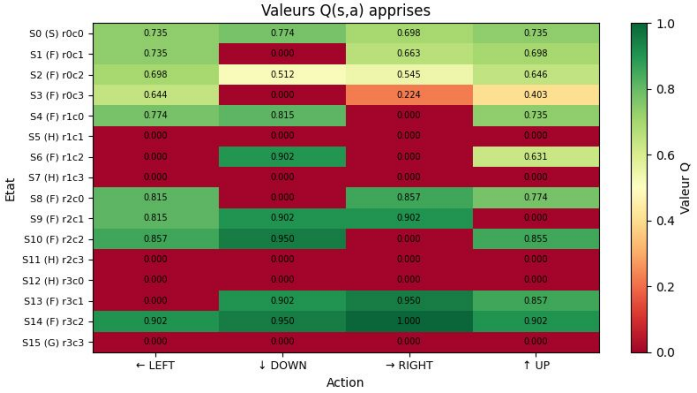
Exercice 2. FrozenLake-v1

Entraînez un agent avec une table de décision (Q-table)

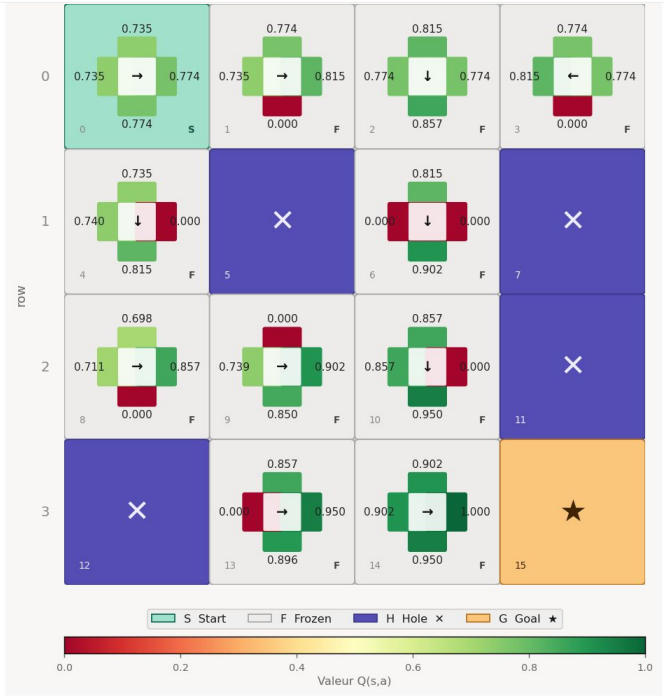
Q-table

État	◂ Gauche	▼ Bas	▸ Droite	▲ Haut
[00]	+0.7351	+0.7738★	+0.6983	+0.7351
[01]	+0.7351★	+0.0000	+0.6634	+0.6983
[02]	+0.6983★	+0.6095	+0.3543	+0.6143
[03]	+0.5134★	+0.0000	+0.0000	+0.0000
[04]	+0.7738	+0.8145★	+0.0000	+0.7351
[05]	+0.0000★	+0.0000	+0.0000	+0.0000
[06]	+0.0000	+0.0000	+0.0000	+0.6634★
[07]	+0.0000★	+0.0000	+0.0000	+0.0000
[08]	+0.8145	+0.0000	+0.8574★	+0.7738
[09]	+0.8145	+0.9025★	+0.9025	+0.0000
[10]	+0.8574	+0.9500★	+0.0000	+0.6237
[11]	+0.0000★	+0.0000	+0.0000	+0.0000
[12]	+0.0000★	+0.0000	+0.0000	+0.0000
[13]	+0.0000	+0.9025	+0.9500★	+0.8574
[14]	+0.9025	+0.9500	+1.0000★	+0.9025
[15]	+0.0000★	+0.0000	+0.0000	+0.0000

Q-table heatmap



Q-Directional Map



⭐*Nouveau outil de visualisation*

Après recherche et consultation de la bibliographie standard en RL (et des documentations web, ia, papers, etc), aucun graphique similaire n'a été répertorié.
Cette implémentation est proposée comme une contribution originale.
Cette fonction introduit une approche inédite pour visualiser l'apprentissage par renforcement : elle cartographie graphiquement la convergence de la Q-table sous forme de vecteurs directionnels.

ASTRODYNAMICS

2

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Entraînez un agent avec une table de décision (Q-table)

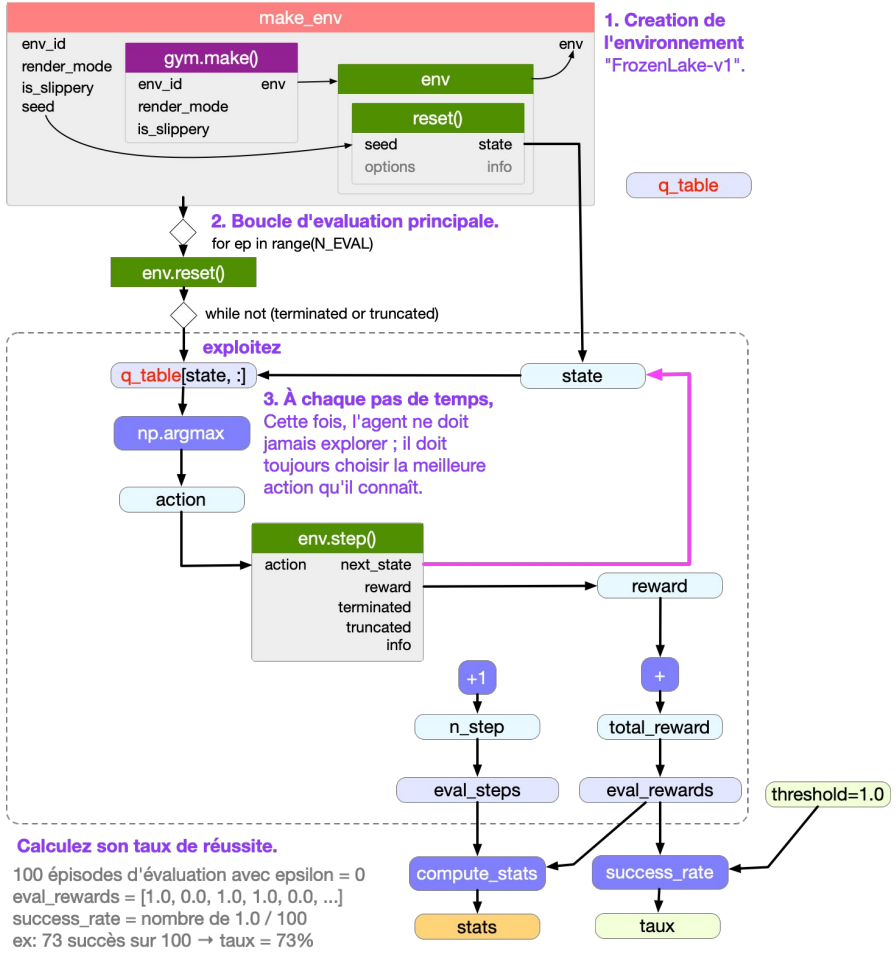
Boucle d'évaluation

is_slippery=True (défaut)
Chaque action a seulement 1/3 de probabilité d'aller dans la direction voulue
L'environnement est stochastique.
Même la politique optimale théorique ne dépasse pas ~74% de succès sur FrozenLake-v1 avec is_slippery=True.
Un taux de 4% reflète une convergence prématurée d'epsilon plutôt que la qualité intrinsèque de la Q-table.

L'action choisie est ***is_slippery=False***
toujours exécutée
*L'****environnement est déterministe.***
100% de succès → la Q-table a appris une politique correcte, mais adaptée à un environnement sans aléa.

Conclusion
Pour obtenir ~70-75% avec is_slippery=True, il faudrait explorer plus longtemps : augmenter N_EPISODES ou réduire EPSILON_DEC (ex: 0.9995 au lieu de 0.998).

```
==================================================
  Evaluation Q-Learning -- 100 episodes -- FrozenLake-v1
==================================================
  Episodes............: 100
  Recompense moyenne..:    1.00  +-  0.00
  Recompense min/max..:    1.00  /   1.00
  Mediane.............:    1.00
  Q25 / Q75...........:    1.00  /   1.00
  Pas moyens..........:     6.0  +-  0.0
  Pas maximum.........:       6
==================================================
  Taux de succes.....: 100.0%
```



ASTRODYNAMICS

2

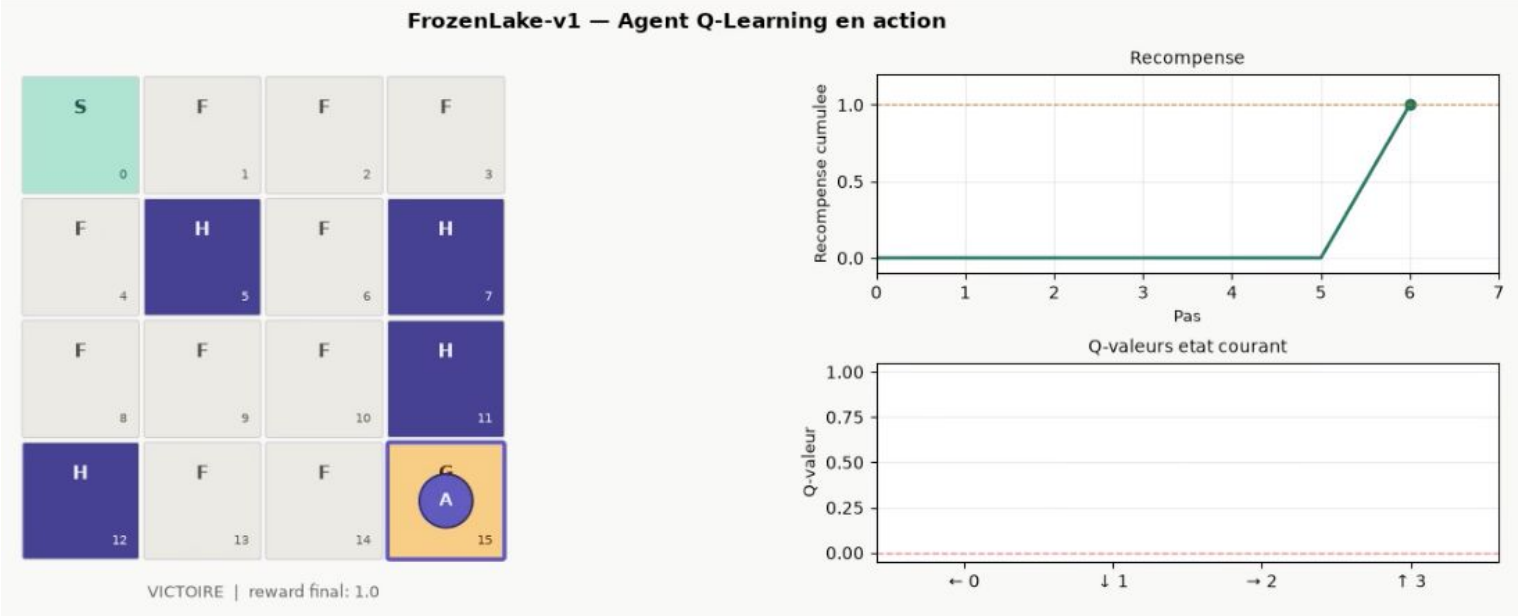
Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Entraînez un agent avec une table de décision (Q-table)



Remplacez la Q-table par un cerveau neuronal (DQN)

CartPole-v1

Espace d'Observation
4 variables continues (Box)

obs[0] x position du chariot
Box [−4.8, +4.8] m

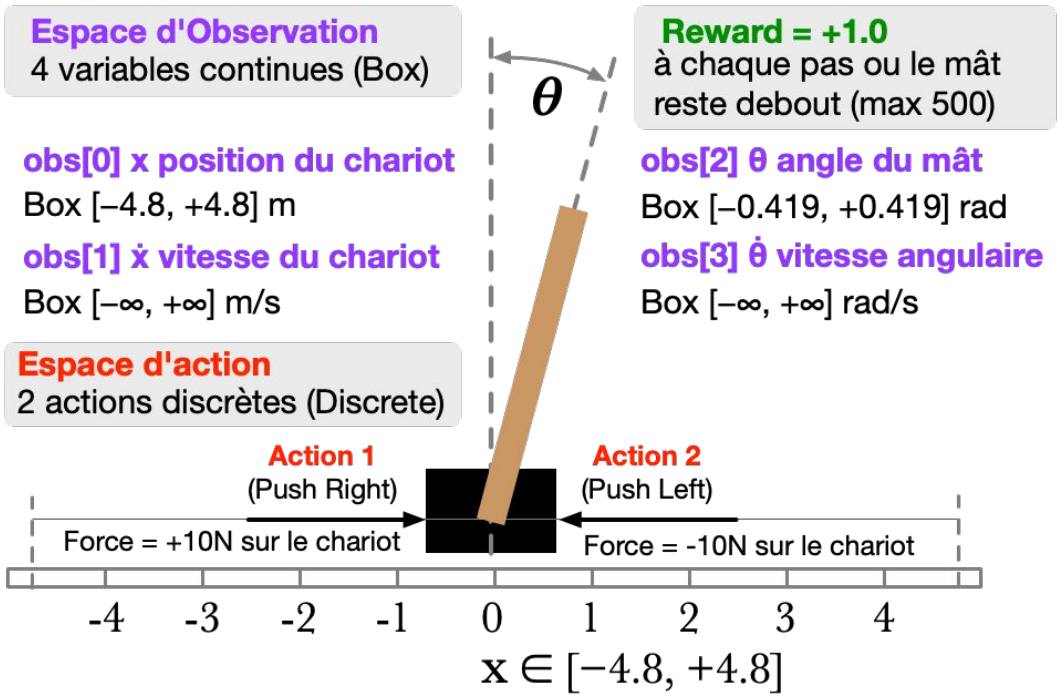
obs[1] ẋ vitesse du chariot
Box [−∞, +∞] m/s

Espace d'action
2 actions discrètes (Discrete)

Reward = +1.0
à chaque pas ou le mât reste debout (max 500)

obs[2] θ angle du mât
Box [−0.419, +0.419] rad

obs[3] θ̇ vitesse angulaire
Box [−∞, +∞] rad/s



Remplacez la Q-table par un cerveau neuronal (DQN)

3

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

DQN

Réseau de neurones Q(s,a)

MLP 3 couches Linear + ReLU, remplace la Q-table pour états continus

nn.Linear → ReLU → Linear → ReLU → Linear

Experience Replay

Buffer de transitions (s,a,r,s',done), brise les corrélations temporelles

deque(maxlen=buffer_size)

Target network

Réseau cible gelé mis à jour périodiquement, stabilise les cibles Q

target_net.load_state_dict(policy_net)

Décroissance ε linéaire

SB3 décroît ε linéairement sur exploration_frac × total_timesteps

exploration_frac = 0.25

Hyperparamètres génériques

- alpha / learning_rate 1e-3
- gamma 0.99
- epsilon_start 1.0
- epsilon_min / final 0.05
- n_épisodes / timesteps 500 000 steps
- batch_size 64
- seed 42

Hyperparamètres spécifiques

- buffer_size 100 000
- learning_starts 1 000
- target_update_interval 5 000
- exploration_frac 0.25
- exploration_final 0.05

ASTRODYNAMICS

3

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

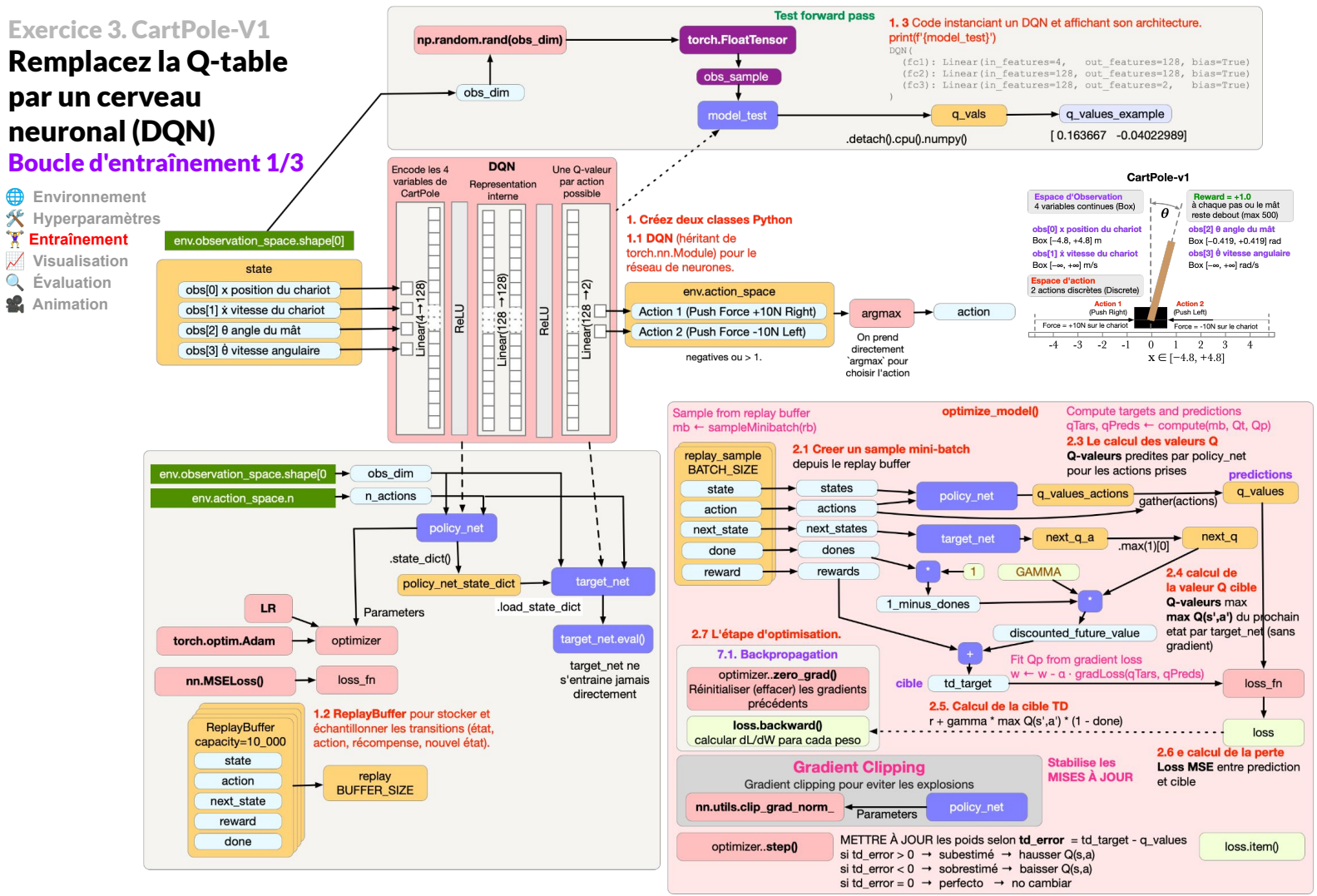
Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Exercice 3. CartPole-V1

Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'entraînement 1/3

- Environnement
- Hyperparamètres
- Entraînement
- Visualisation
- Évaluation
- Animation



ASTRODYNAMICS

3

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Exercice 3. CartPole-V1

Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'entraînement 2/3

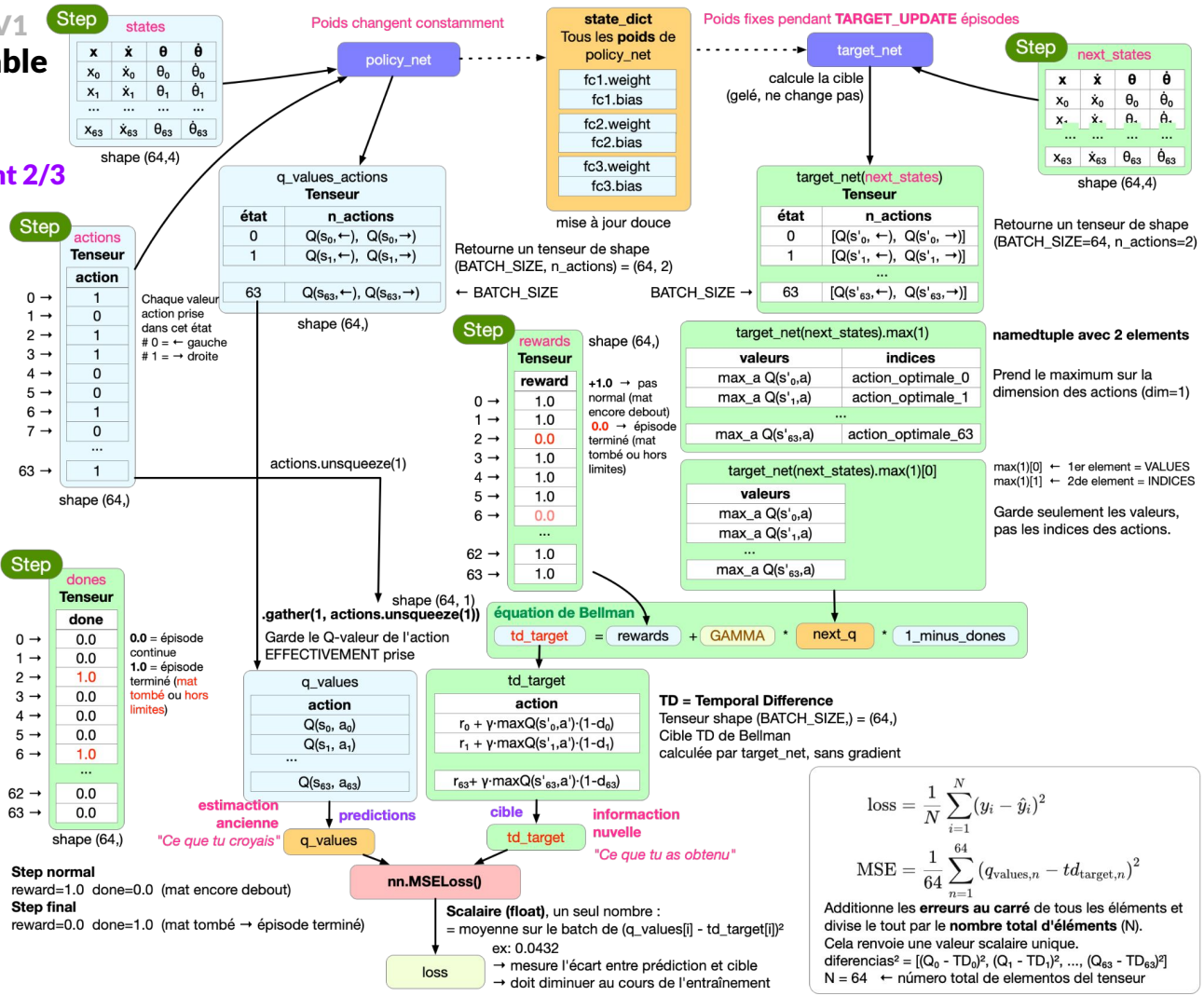
🌐 Environnement
🛠️ Hyperparamètres
🏋️ Entraînement
📈 Visualisation
🔍 Évaluation
🎥 Animation

DQN (Deep Q-Network)
Calcul de la Perte (Loss)

Repérez où le policy_net est utilisé pour prédire les ***Q-values des actions prises****.*

Repérez où le target_net est utilisé pour estimer la valeur maximale des états suivants.
C'est un point clé.

Comprenez comment la loss est calculée entre ces deux quantités.



ASTRODYNAMICS

3

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Exercice 3. CartPole-V1

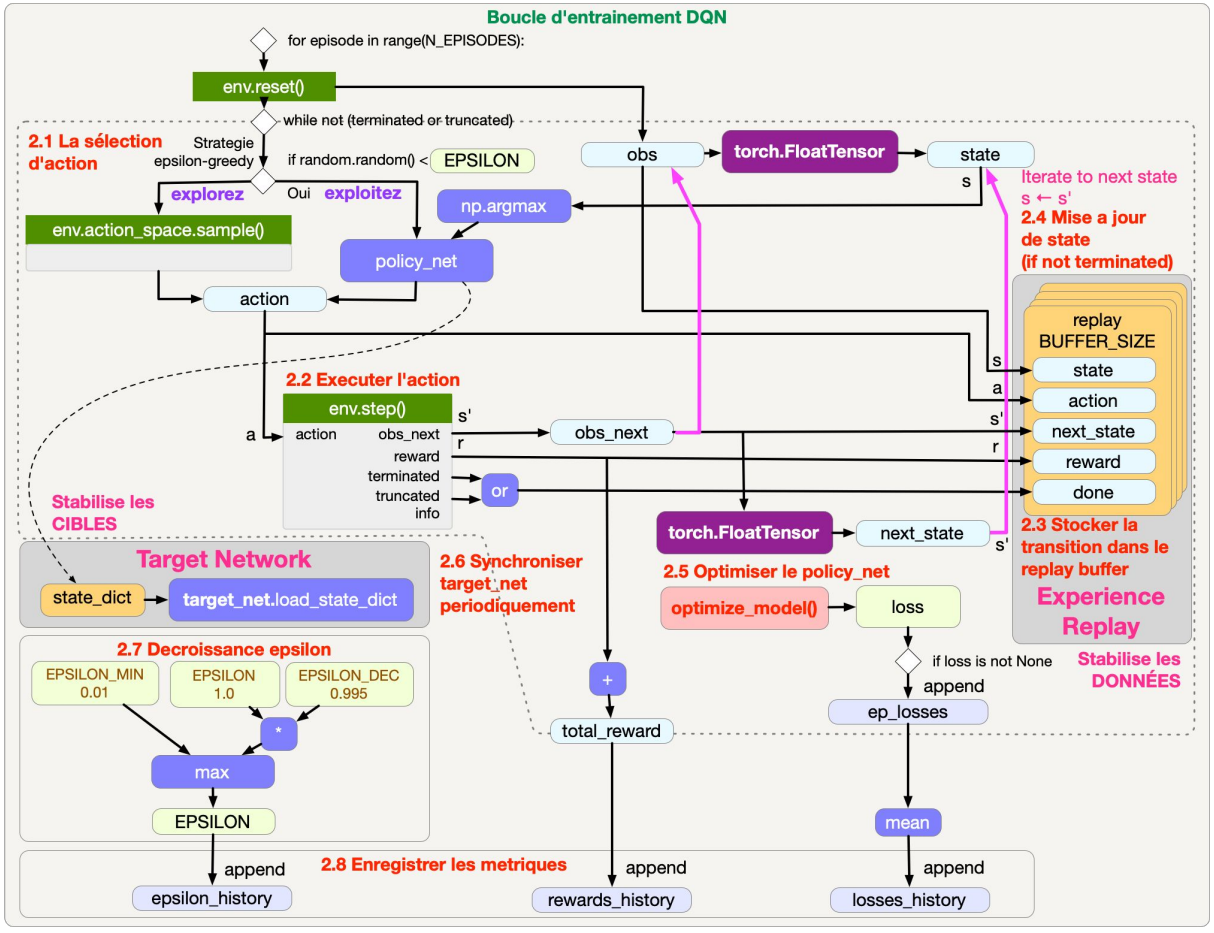
Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'entraînement 3/3

- Environnement
- Hyperparamètres
- Entraînement
- Visualisation
- Évaluation
- Animation

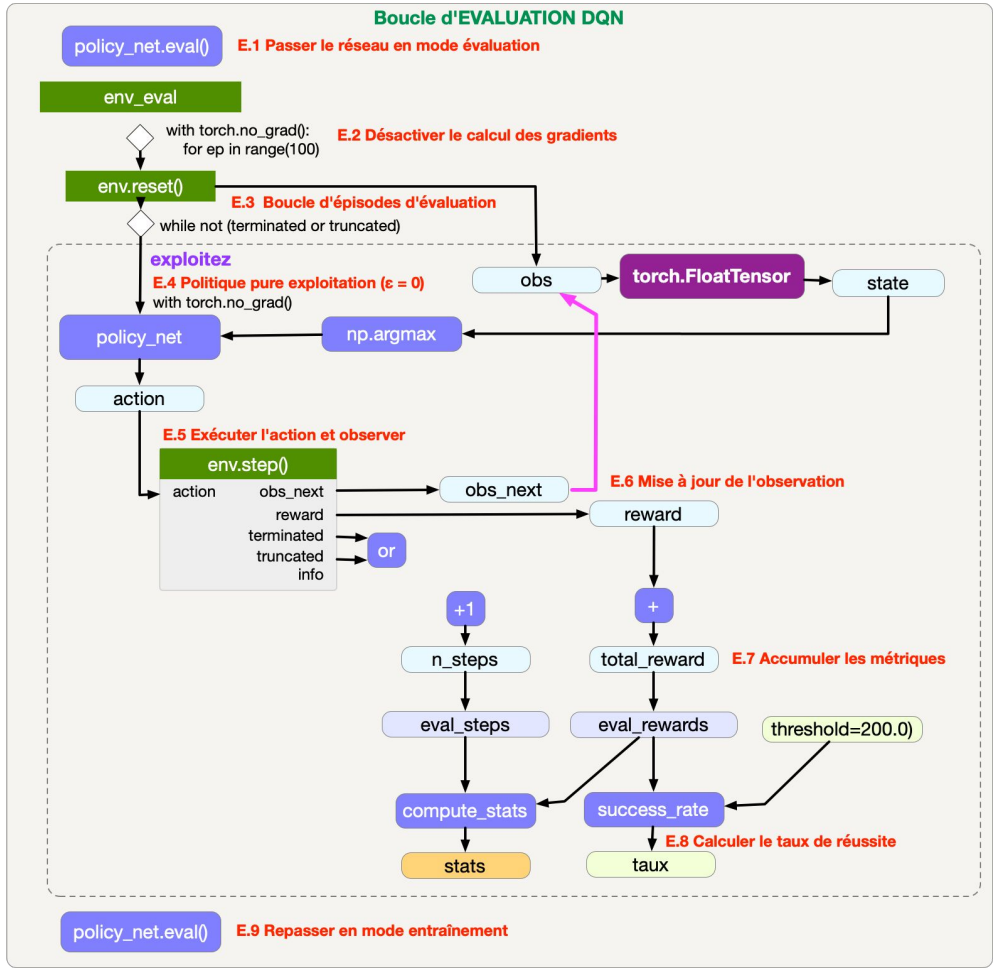
Analyser la boucle, la comprendre et la faire fonctionner.

Saisir la logique de chaque bloc :
- *la sélection d'action,*
- *le calcul des valeurs Q,*
- *le calcul de la valeur Q cible (avec le target network),*
- *le calcul de la perte, et*
- *l'étape d'optimisation.*



Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'évaluation



Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'entraînement. Avec Stable-Baselines3

ASTRODYNAMICS

3

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

CartPole-v1

gym.make()

env_sb3

stable_baselines3

stable_baselines3 CNN

Obligatoires (toujours présents)

'MlpPolicy' (D)
architecture du réseau. MlpPolicy (vecteurs) ou CnnPolicy (images)

env_sb3, (B)
environnement Gymnasium

Essentiels

learning_rate = 1e-3
gamma = 0.99
batch_size = 64
uffer_size = 50_000
learning_starts = 1_000
target_update_interval = 500
verbose = 1
tensorboard_log = './logs_sb3/'
exploration_fraction = 0.3
exploration_final_eps = 0.01

Apprentissage
learning_rate = 1e-3, (D)
α Adam — défaut SB3 : 1e-4
gamma = 0.99, (D)
γ facteur d'actualisation
batch_size = 64, (D)
taille du mini-batch

Replay buffer
buffer_size = 50_000, (G) (C)
capacité du buffer
learning_starts = 1_000, (C)
MIN_BUFFER. nombre de pas avant le début de l'apprentissage

Réseau cible
target_update_interval = 500, (E)
synchronisation du target_net tous les N pas

Journaux (logs)
verbose = 1,
0=silencieux, 1=infos, 2=debug
tensorboard_log
= './logs_sb3/',

Exploration
exploration_fraction = 0.3, (F)
% des timesteps pendant lesquels ε décroît
exploration_final_eps = 0.01, (F)
ε_min final

Utiles (à connaître pour optimiser)

Fréquence d'entraînement
train_freq = 4,
optimisation tous les N pas (défaut
gradient_steps = 1,
nombre de descentes de gradient par rollout (-1 = automatique)

Exploration complète
exploration_initial_eps = 1.0, (A)
ε initial (défaut)

Stabilité
max_grad_norm = 10,
clipping des gradients (défaut SB3)
tau = 1.0,
1.0 = mise à jour dure, <1 = mise à jour douce (Polyak)

Reproductibilité
seed
= 42
device = 'auto',
GPU si disponible

Avancés (rarement nécessaires)

Retours N-step
n_steps = 1,
>1 = apprentissage Q-Learning à N pas

Mémoire
optimize_memory_usage = False,
buffer plus efficace mais plus complexe

Replay buffer personnalisé
replay_buffer_class = None,
HerReplayBuffer pour les environnements à objectifs
replay_buffer_kwargs
= None

Retours N-step
_init_setup_model = True,
ne pas modifier

Logs avancés
stats_window_size = 100,
fenêtre de moyenne des récompenses dans les logs
policy_kwargs = None,
net_arch, activation_fn personnalisés

```
DQNPolicy(
  (q_net): QNetwork(
    (features_extractor): FlattenExtractor(
      (flatten): Flatten(start_dim=1, end_dim=-1)
    )
    (q_net): Sequential(
      (0): Linear(in_features=4, out_features=64, bias=True)
      (1): ReLU()
      (2): Linear(in_features=64, out_features=64, bias=True)
      (3): ReLU()
      (4): Linear(in_features=64, out_features=2, bias=True)
    )
  )
  (q_net_target): QNetwork(
    (features_extractor): FlattenExtractor(
      (flatten): Flatten(start_dim=1, end_dim=-1)
    )
    (q_net): Sequential(
      (0): Linear(in_features=4, out_features=64, bias=True)
      (1): ReLU()
      (2): Linear(in_features=64, out_features=64, bias=True)
      (3): ReLU()
      (4): Linear(in_features=64, out_features=2, bias=True)
    )
  )
)
```

Show Architecture

model_sb3

model_sb3.policy

Entrainement

model_sb3.learn

Géré automatiquement par SB3
A. action ε-greedy auto
B. env.step() obs,reward,done
C. replay.push() Experience Replay
D. optimize_model() MSE + backprop
E. target_net sync chaque 500 steps
F. ε decay (linéaire sur exploration_fraction × steps
G. Compte les pas env.step()
50 000 steps ≈
250 épisodes × 200 pas/épisode

```
100% ━━…━━ 49,889/50,000
[ 0:00:59 < 0:00:01 , 800 it/s ]
...
-------------------------------------
| rollout/                |         |
|    ep_len_mean          | 132     |
|    ep_rew_mean          | 132     |
|    exploration_rate     | 0.01    |
| time/                   |         |
|    episodes             | 492     |
|    fps                  | 841     |
|    time_elapsed         | 59      |
|    total_timesteps      | 49960   |
| train/                  |         |
|    learning_rate        | 0.001   |
|    loss                 | 1.07    |
|    n_updates            | 12239   |
-------------------------------------
Entrainement SB3 termine.
```

model_sb3
agent entraîné

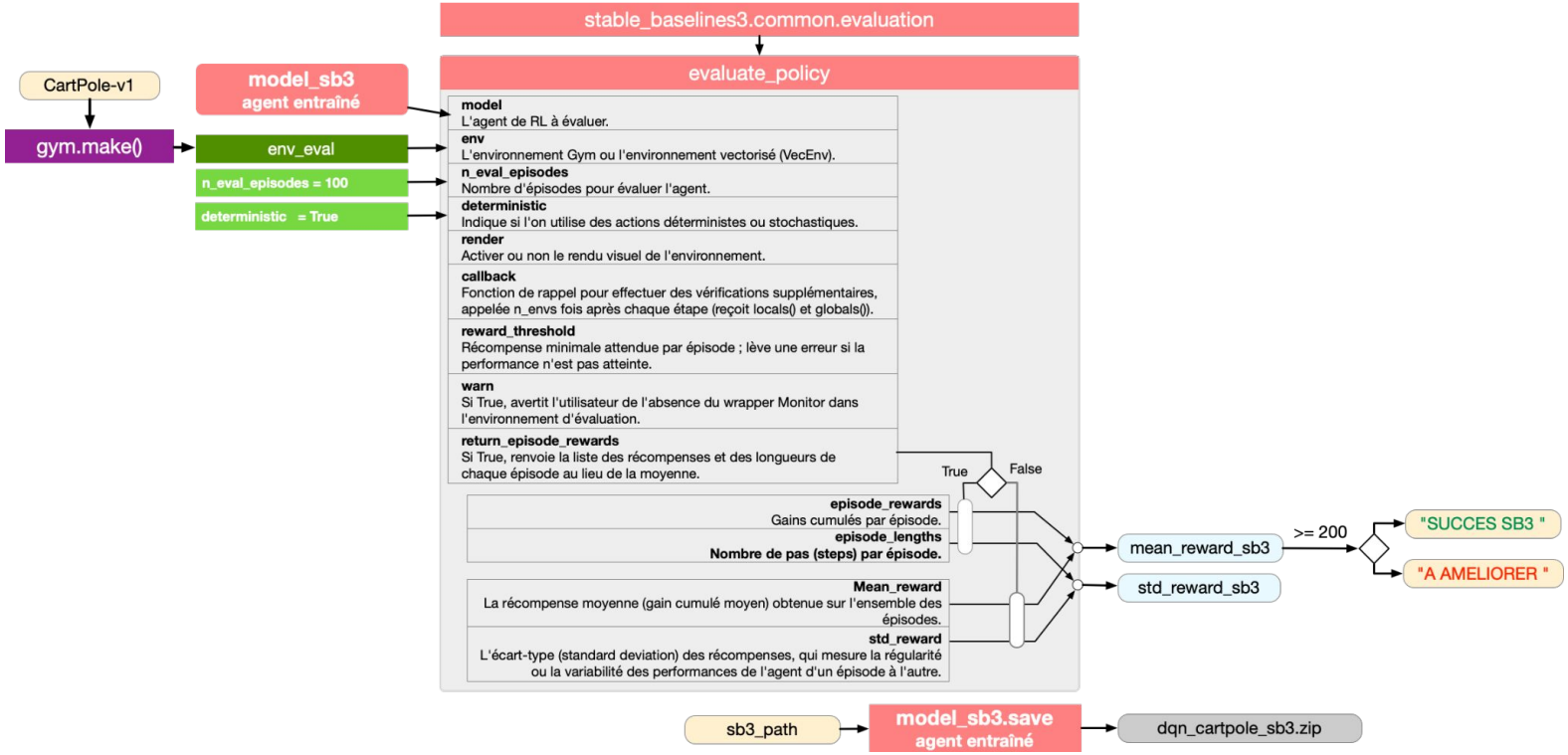
policy_net
- Utilisé pour model.predict() et evaluate_policy().
- Il contient les poids appris
- C'est le "cerveau" de l'agent.

target_net
- Reste présent en memoire mais n'est plus utilisé en inférence.
- Sa dernière synchronisation avec policy_net a eu lieu pendant l'entraînement.
- En pratique ses poids sont quasi-identiques à ceux de policy_net à la fin.

replay buffer
- Reste en mémoire mais n'est plus alimenté.
- Il contient les dernières 50_000 transitions de l'entraînement.

Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'évaluation. Avec Stable-Baselines3 (evaluate_policy)



3

Introduction

Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

Remplacez la Q-table par un cerveau neuronal (DQN)

Boucle d'évaluation. Avec Stable-Baselines3 (Boucle explicite pour métriques détaillées)

ASTRODYNAMICS

3

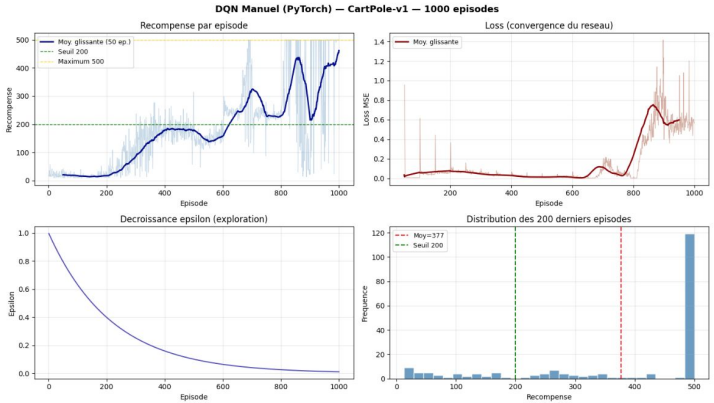
Introduction

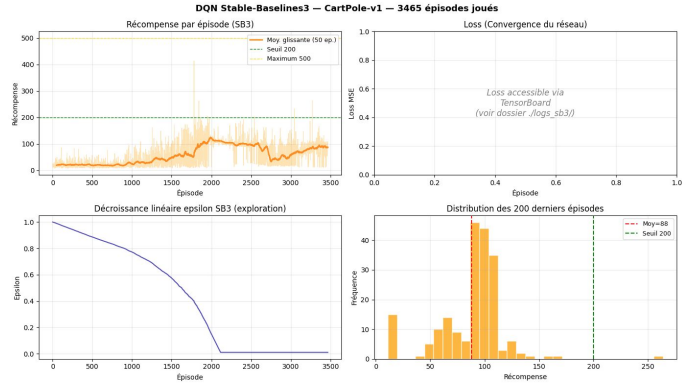
Exercice 1
- Environnement
- Politique aléatoire
- Interprétation

Exercice 2
- Environnement
- Boucle d'Entraînement
- Visualisation
- Boucle de Validation
- Visualisation

Exercice 3
- Environnement
- Manual con PyTorch
- Stable-Baselines3

DQN Manuel — Essai 1 — DQN DB3





DQN Manuel — Essai 2 — DQN DB3

